

Image processing

Lecture - 2

Assaf B. Spanier

Lecture topic -
Image Enhancement
Techniques

Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Image Histogram	.5
Otus method	.6
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

Normalization (image processing)

In image processing, **normalization** is a process that changes the range of pixel intensity values. Applications include photographs with poor contrast due to glare, for example. Normalization is sometimes called contrast stretching or histogram stretching. In more general fields of data processing, such as digital signal processing, it is referred to as dynamic range expansion.^[1]

Example



Example



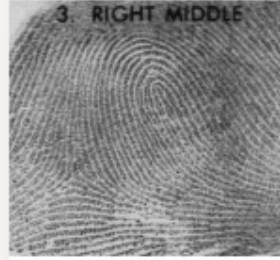
M89_f0104_08
PNG - 206.19 KB



M89_f0105_03
PNG - 212.32 KB



M89_f0106_08
PNG - 208.26 KB



M89_f0107_03
PNG - 204.71 KB



M89_f0108_03
PNG - 200.44 KB



M89_f0109_05
PNG - 176.9 KB



M89_f0110_08
PNG - 206.68 KB



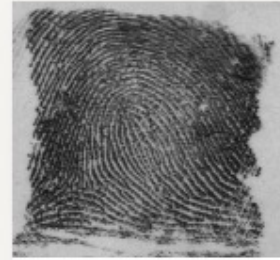
M89_f0111_02
PNG - 200.19 KB



M89_f0112_02
PNG - 199.25 KB



M89_f0113_01
PNG - 204.17 KB



M89_f0114_05
PNG - 205.3 KB



M89_f0115_03
PNG - 216.29 KB



M89_f0116_06



M89_f0117_09



M89_f0118_08



M89_f0119_08



M89_f0120_08



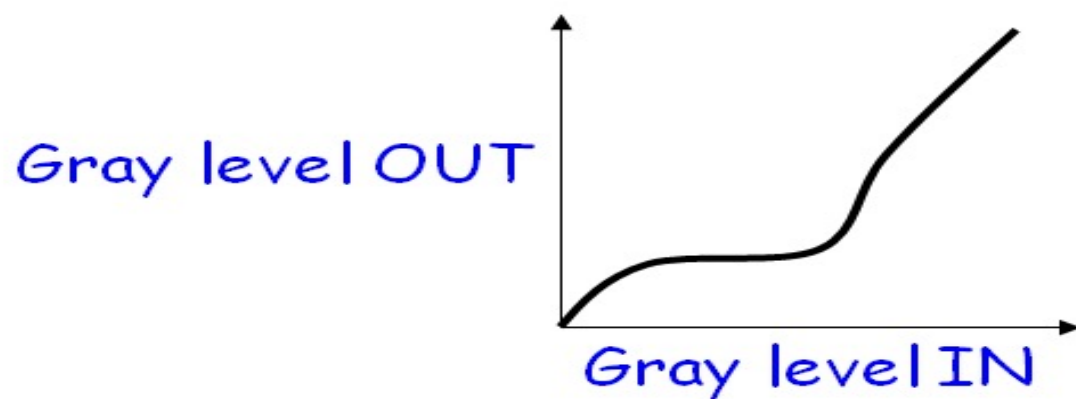
M89_f0121_07

Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Image Histogram	.5
Otus method	.6
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

פעולות על דרגות האפור

טכניספורמזיה על דרגות אפור ללא התייחסות לכמות או מיקום בתמונה.



Gray level IN	0	1	2	3	4	5	...	255
Gray level OUT	0	0	0	2	50	51	...	215

Gray Level In (GLI) and Gray Level Out (GLO) functions

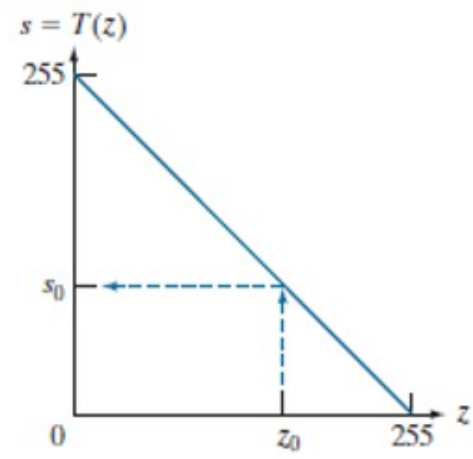
```
1 import numpy as np
2
3 date_point = [2, 4, 4, 4, 6]
4 GLI = [2, 4, 6]
5 GLO = [1, 3, 5]
6
7 gfg = np.interp(date_point, GLI, GLO)
8
9 print(gfg)
```


Gray Level In (GLI) and Gray Level Out (GLO) functions - Second Example

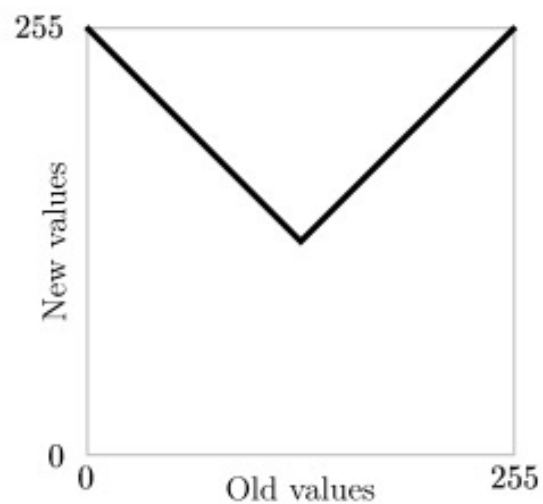
```
# Create an image with a non-uniform intensity distribution
image = np.zeros((256, 256), dtype=np.uint8)
image[128:256, 128:256] = 255
```

```
GLI = [ 0 , 255]
GLO = [255, 0]
gray_level_out = np.interp(image.flatten(), [ 0 , 255], [255, 0])
gray_level_out_image = gray_level_out.reshape(image.shape)
```

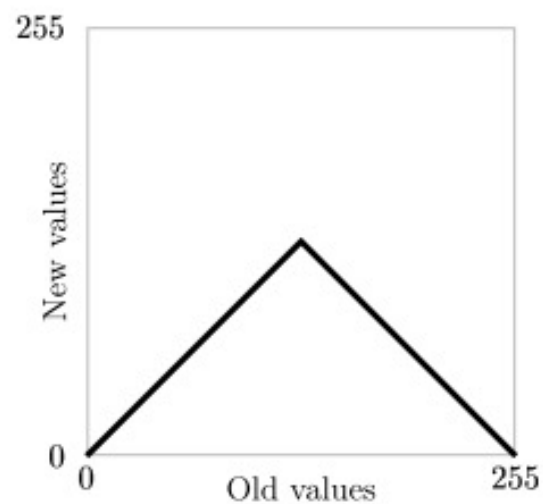
גדטר



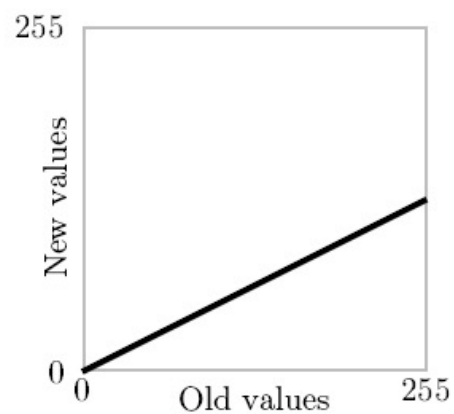
עוד ועוד אפשרויות...



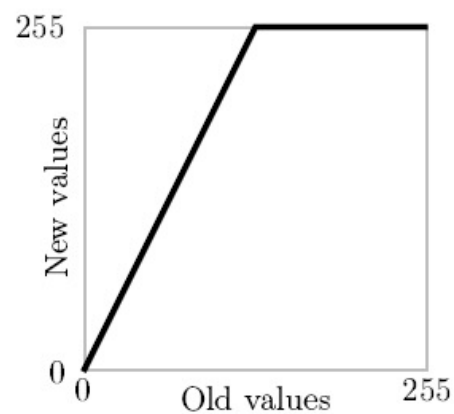
Complementing only dark pixels



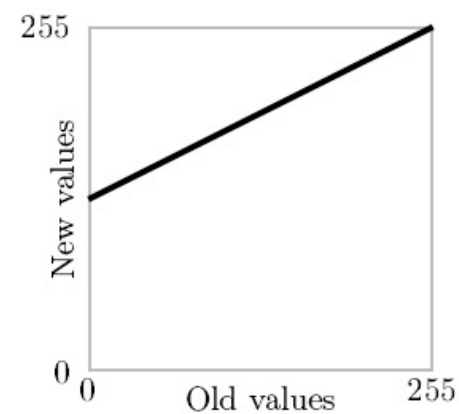
Complementing only light pixels



$y = x/2$



$y = 2x$



$y = x/2 + 128$

Figure 2.4: Using multiplication and division

Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Image Histogram	.5
Otus method	.6
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

Brightness

- The brightness is the intensity
- Change brightness: $g(x,y) = f(x,y) + b$
 - To each pixel is added the value b
 - $f(x,y)$ is the input image
 - $g(x,y)$ is the (enhanced) output image
- If $b > 0 \Rightarrow$ brighter image
- If $b < 0 \Rightarrow$ less bright image



less bright image

brighten_image

```
import numpy as np
import matplotlib.pyplot as plt

def brighten_image(image_path, brightness_factor):
    # Read the image using Matplotlib (as it directly reads RGB images)
    image = plt.imread(image_path)

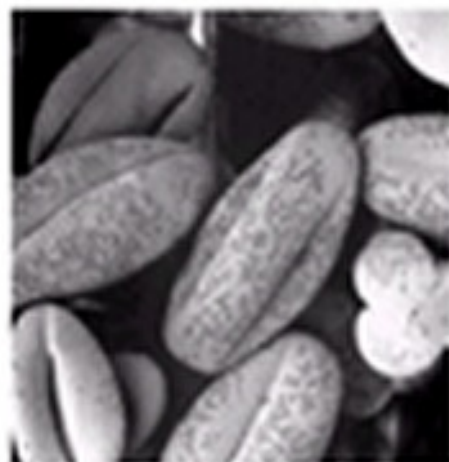
    # Clip values to be in the valid range [0, 1]
    brightened_image = np.clip(image + brightness_factor, 0, 1)

    # Display the result (optional)
    plt.imshow(brightened_image)
    plt.show()

    # Save the result (optional)
    plt.imsave("brightened_image_numpy.jpg", brightened_image)
```


Contrast

- The contrast describes the level of details we can see
- Change contrast: $g(x,y) = a * f(x,y)$
- Each pixel is multiplied by a
 - $f(x,y)$ is the input image
 - $g(x,y)$ is the (enhanced) output image
- If $a > 1 \Rightarrow$ more contrast
- If $a < 1 \Rightarrow$ less contrast



more contrast

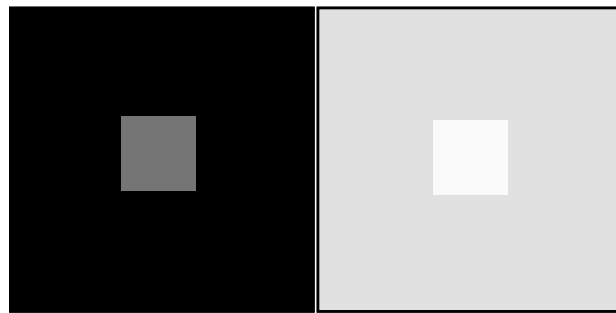


less contrast

Image Contrast

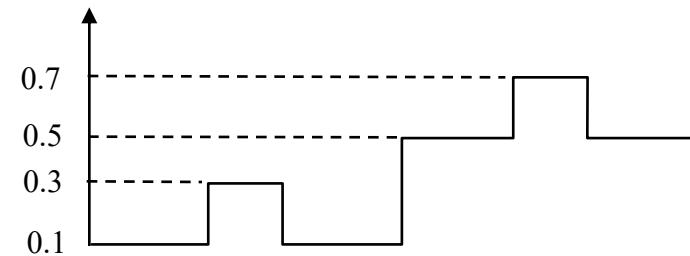
- The **local contrast** at an image point denotes the (relative) difference between the intensity of the point and the intensity of its neighborhood:

$$C = \left| \frac{I_p - I_n}{I_n} \right|$$



$$C = \left| \frac{0.3 - 0.1}{0.1} \right| = 2$$

$$C = \left| \frac{0.7 - 0.5}{0.5} \right| = 0.4$$

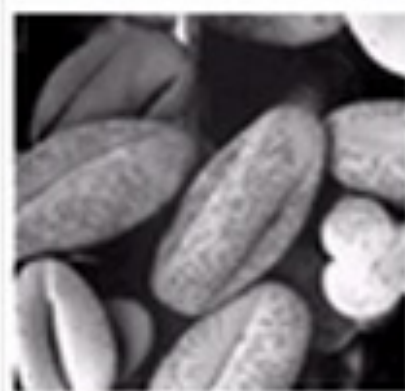


Contrast

```
def increase_contrast(image_path, contrast_factor):  
    # Read the image using Matplotlib  
    image = plt.imread(image_path,0)  
  
    # Adjust contrast  
    high_contrast_image = (image) * contrast_factor  
  
    # Clip values to be in the valid range [0, 1]  
    high_contrast_image = np.clip(high_contrast_image, 0, 1)  
  
    # Display the result (optional)  
    plt.imshow(high_contrast_image)  
    plt.show()
```

- Grey level enhancement
 - Process one pixel at a time independent of all other pixels
 - For example used to correct Brightness and Contrast (remote control)

Correct



Too high
brightness



Too low
brightness



Too high
contrast



Too low
contrast

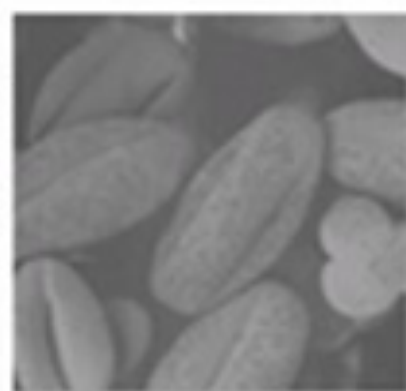
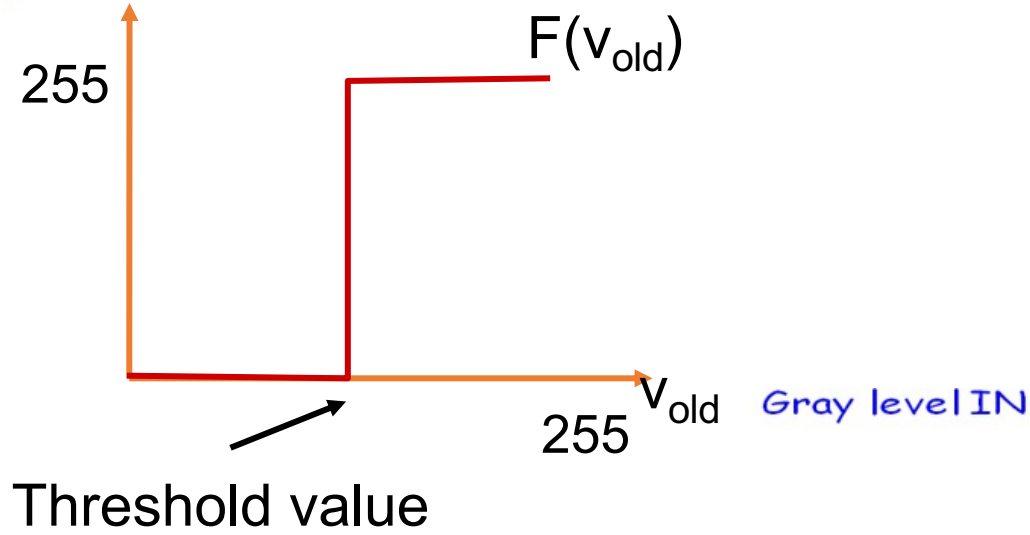


Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Image Histogram	.5
Otus method	.6
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

מקרה קיצוני הוא f לקיחת T (threshold).
סוב לציגו עוצמים.

Gray level OUT



threshold_image_with_interp

```
def threshold_image_with_interp(image_array, threshold_value):  
    # Use np.interp for thresholding  
    # Pixels below the threshold are set to 0, and above the threshold to 255  
    binary_image = np.interp(image_array, [0, threshold_value, 255], [0, 0, 255])  
  
    return binary_image
```


cv2.threshold

1. **src**: The source image,.
2. **thresh**: The threshold value.
3. **maxval**: The maximum value to be used with the THRESH_BINARY and THRESH_BINARY_INV thresholding types..
4. **type**: The type of thresholding to be applied
 1. cv2.THRESH_BINARY: Pixels that are greater than the specified threshold value are set to maxval, the rest are set to 0.
 2. cv2.THRESH_BINARY_INV: Inverse of THRESH_BINARY. Pixels that are below the threshold value are set to maxval, others are set to 0.
 3. cv2.THRESH_TRUNC: Pixels that are greater than the threshold are set to the threshold value, the rest remain unchanged.
 4. cv2.THRESH_TOZERO: Pixels that are below the threshold are set to 0, the rest remain unchanged.
 5. cv2.THRESH_TOZERO_INV: Inverse of THRESH_TOZERO. Pixels above the threshold are set to 0, the rest remain unchanged.

```
# import opencv
```

```
import cv2
```

```
# Read image
```

```
src = cv2.imread("threshold.png",  
cv2.IMREAD_GRAYSCALE);
```

```
# Basic threshold example
```

```
th, dst = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-threshold-example.jpg", dst);
```

```
# Thresholding with maxVal set to 128
```

```
th, dst = cv2.threshold(src, 0, 128, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary-maxval.jpg", dst);
```

```
# Thresholding with threshold value set 127
```

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary.jpg", dst);
```



```
# import opencv
```

```
import cv2
```

```
# Read image
```

```
src = cv2.imread("threshold.png",  
cv2.IMREAD_GRAYSCALE);
```

$$\text{dst}(x, y) = \begin{cases} \text{maxVal} & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$

```
# Basic threshold example
```

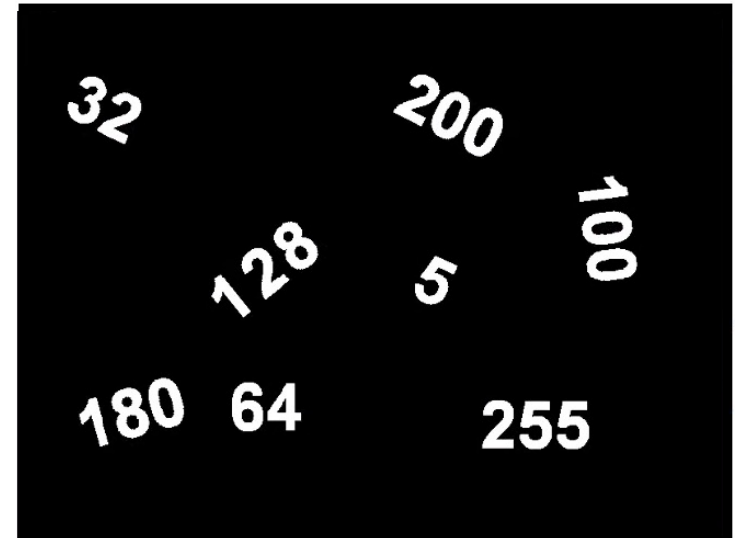
```
th, dst = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-threshold-example.jpg", dst);
```

```
# Thresholding with maxVal set to 128
```

```
th, dst = cv2.threshold(src, 0, 128, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary-maxval.jpg", dst);
```

```
# Thresholding with threshold value set 127
```

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary.jpg", dst);
```



```
# import opencv
```

```
import cv2
```

```
# Read image
```

```
src = cv2.imread("threshold.png",  
cv2.IMREAD_GRAYSCALE);
```

```
# Basic threshold example
```

```
th, dst = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-threshold-example.jpg", dst);
```

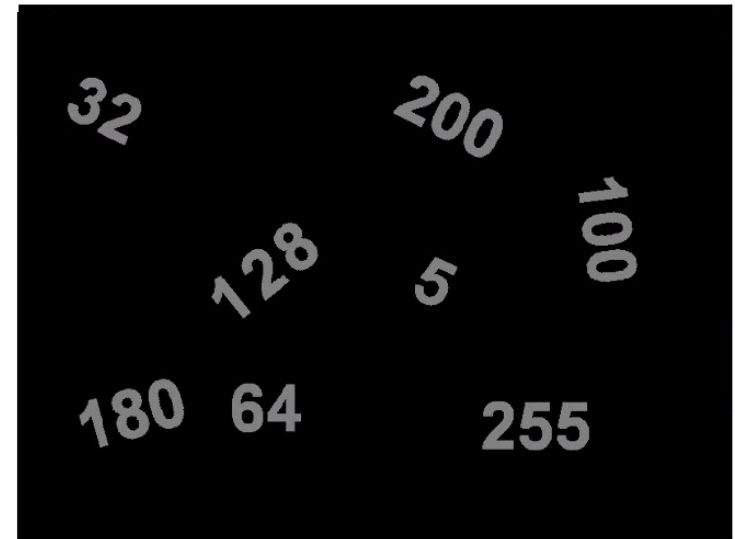
```
# Thresholding with maxVal set to 128
```

```
th, dst = cv2.threshold(src, 0, 128, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary-maxval.jpg", dst);
```

```
# Thresholding with threshold value set 127
```

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary.jpg", dst);
```

$$\text{dst}(x, y) = \begin{cases} \text{maxVal} & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$



```
# import opencv
```

```
import cv2
```

```
# Read image
```

```
src = cv2.imread("threshold.png",  
cv2.IMREAD_GRAYSCALE);
```

```
# Basic threshold example
```

```
th, dst = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-threshold-example.jpg", dst);
```

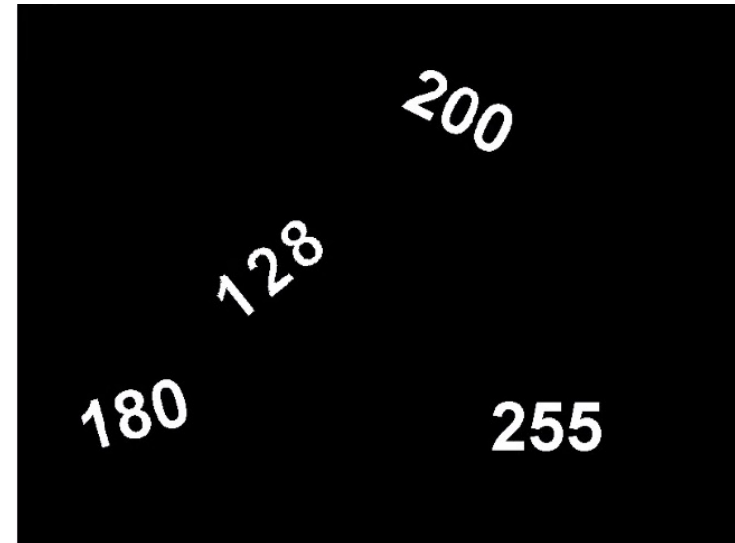
```
# Thresholding with maxVal set to 128
```

```
th, dst = cv2.threshold(src, 0, 128, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary-maxval.jpg", dst);
```

```
# Thresholding with threshold value set 127
```

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_BINARY);  
cv2.imwrite("opencv-thresh-binary.jpg", dst);
```

$$\text{dst}(x, y) = \begin{cases} \text{maxVal} & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$



```
# Thresholding using THRESH_BINARY_INV
```

```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_BINARY_INV);  
cv2.imwrite("opencv-thresh-binary-inv.jpg", dst);
```

$$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{maxVal} & \text{otherwise} \end{cases}$$

```
# Thresholding using THRESH_TRUNC
```

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TRUNC);  
cv2.imwrite("opencv-thresh-trunc.jpg", dst);
```

```
# Thresholding using THRESH_TOZERO
```

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TOZERO);  
cv2.imwrite("opencv-thresh-tozero.jpg", dst);
```

```
# Thresholding using THRESH_TOZERO_INV
```

```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_TOZERO_INV);  
cv2.imwrite("opencv-thresh-to-zero-inv.jpg", dst);
```

180 128 200 255

Thresholding using THRESH_BINARY_INV

```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_BINARY_INV);  
cv2.imwrite("opencv-thresh-binary-inv.jpg", dst);
```

$$\text{dst}(x, y) = \begin{cases} \text{threshold} & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$$

Thresholding using THRESH_TRUNC

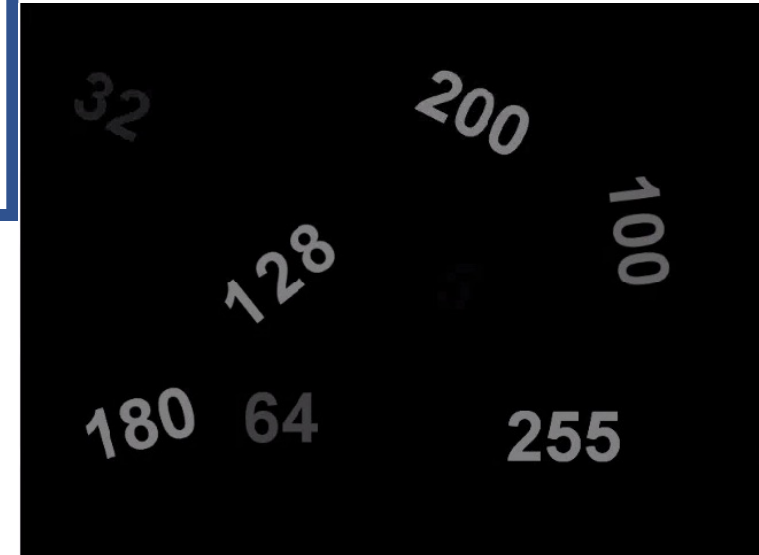
```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TRUNC);  
cv2.imwrite("opencv-thresh-trunc.jpg", dst);
```

Thresholding using THRESH_TOZERO

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TOZERO);  
cv2.imwrite("opencv-thresh-tozero.jpg", dst);
```

Thresholding using THRESH_TOZERO_INV

```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_TOZERO_INV);  
cv2.imwrite("opencv-thresh-to-zero-inv.jpg", dst);
```



Thresholding rule

```
1 | # Truncate Threshold  
2 | if src(x,y) > thresh  
3 |     dst(x,y) = thresh  
4 | else  
5 |     dst(x,y) = src(x,y)
```


Thresholding using THRESH_BINARY_INV

```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_BINARY_INV);  
cv2.imwrite("opencv-thresh-binary-inv.jpg", dst);
```

$$\text{dst}(x, y) = \begin{cases} \text{src}(x, y) & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$

Thresholding using THRESH_TRUNC

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TRUNC);  
cv2.imwrite("opencv-thresh-trunc.jpg", dst);
```

Thresholding using THRESH_TOZERO

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TOZERO);  
cv2.imwrite("opencv-thresh-tozero.jpg", dst);
```

Thresholding using THRESH_TOZERO_INV

```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_TOZERO_INV);  
cv2.imwrite("opencv-thresh-to-zero-inv.jpg", dst);
```



Thresholding using THRESH_BINARY_INV

```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_BINARY_INV);  
cv2.imwrite("opencv-thresh-binary-inv.jpg", dst);
```

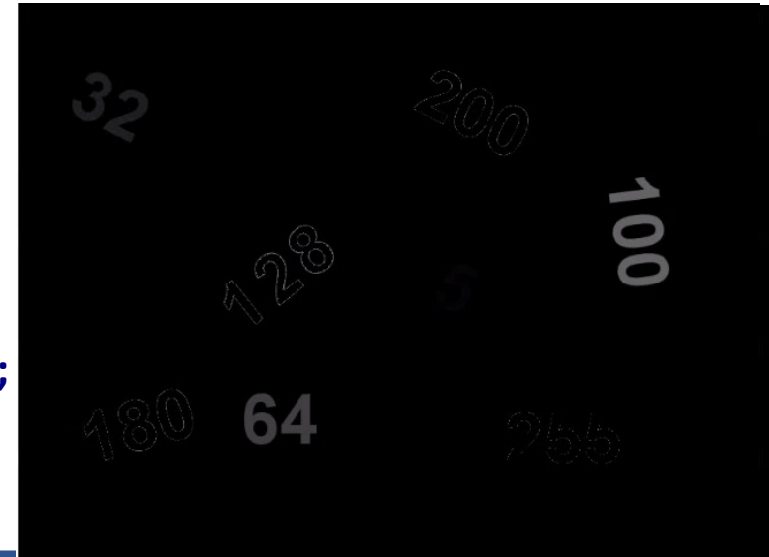
$$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$$

Thresholding using THRESH_TRUNC

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TRUNC);  
cv2.imwrite("opencv-thresh-trunc.jpg", dst);
```

Thresholding using THRESH_TOZERO

```
th, dst = cv2.threshold(src, 127, 255, cv2.THRESH_TOZERO);  
cv2.imwrite("opencv-thresh-tozero.jpg", dst);
```



Thresholding using THRESH_TOZERO_INV

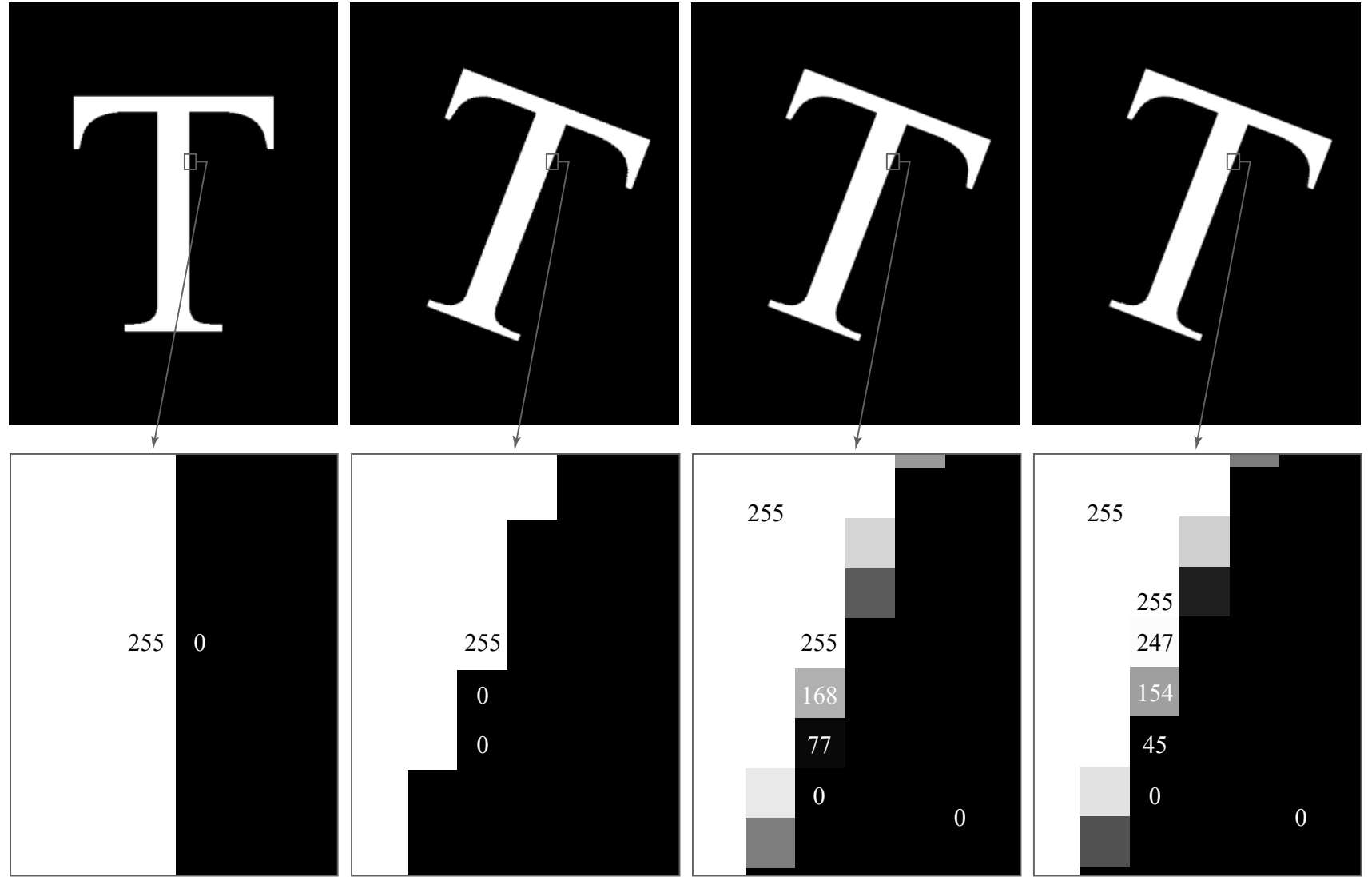
```
th, dst = cv2.threshold(src, 127, 255,  
cv2.THRESH_TOZERO_INV);  
cv2.imwrite("opencv-thresh-to-zero-inv.jpg", dst);
```

Thresholding rule

```
1 | # Inverted Threshold to Zero  
2 | if src(x,y) > thresh  
3 |     dst(x,y) = 0  
4 | else  
5 |     dst(x,y) = src(x,y)
```

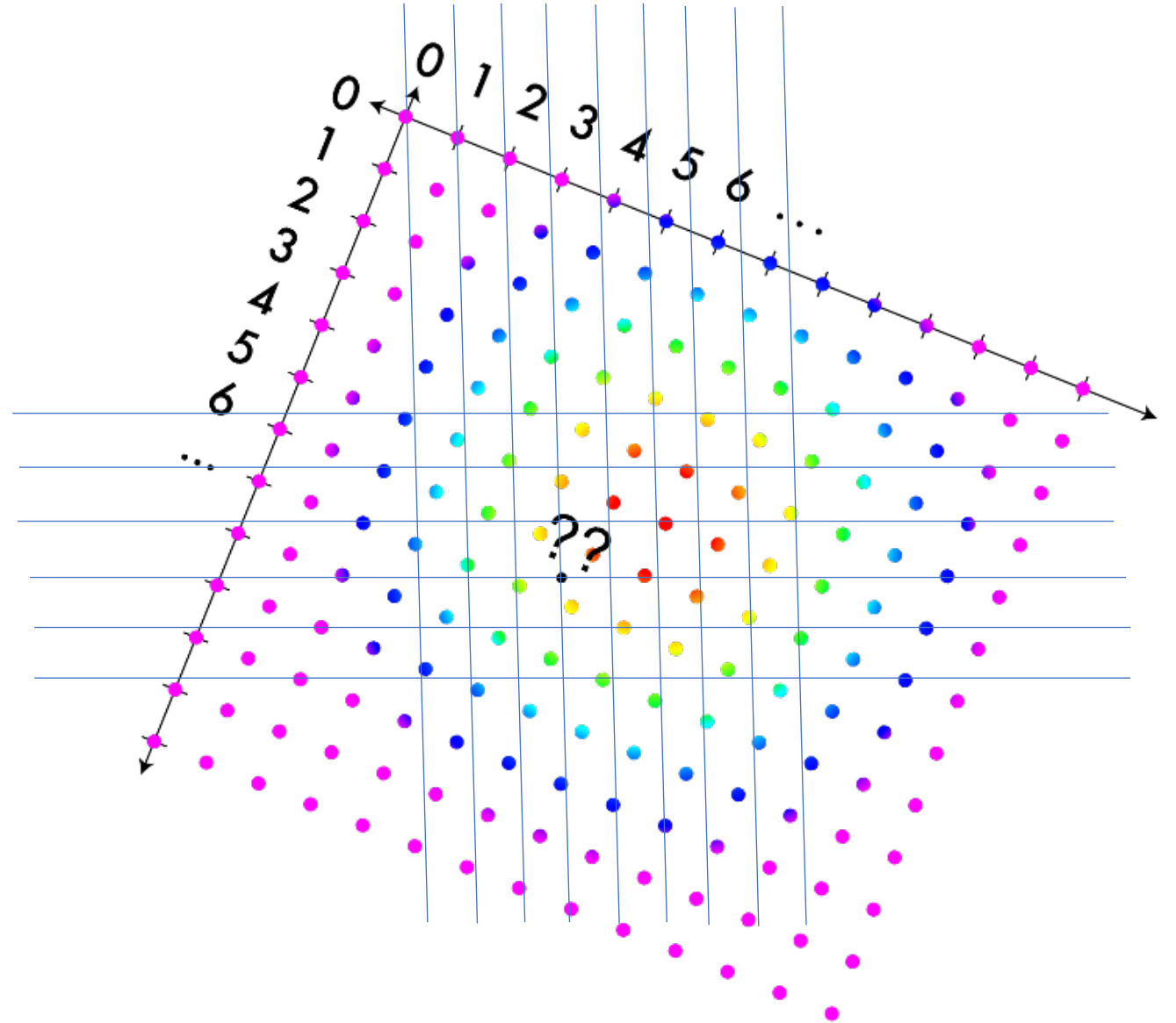
Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Image Histogram	.5
Otus method	.6
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4



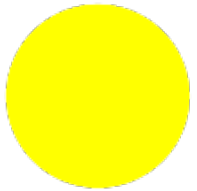
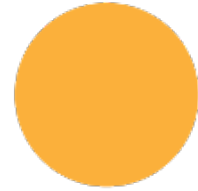
Bilinear interpolation: for grids, pretty good

This time find the closest pixels
in a box



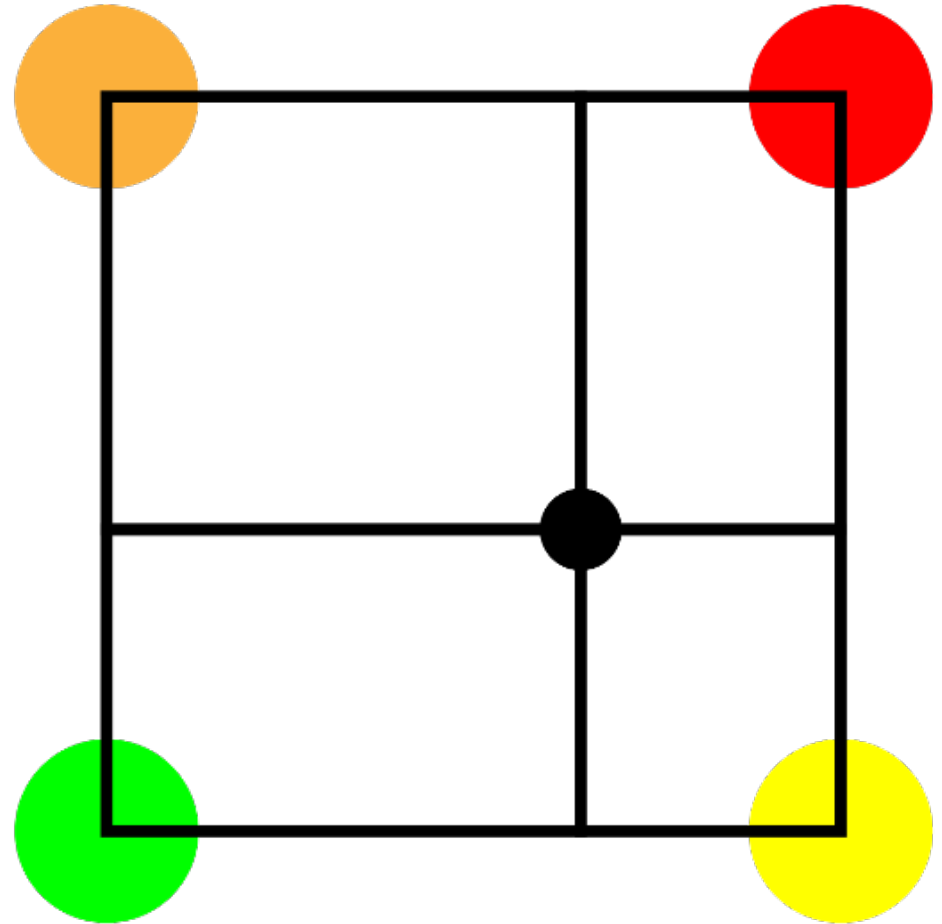
Bilinear interpolation: for grids, pretty good

This time find the closest pixels
in a box



Bilinear interpolation: for grids, pretty good

This time find the closest pixels
in a box



Bilinear interpolation: for grids, pretty good

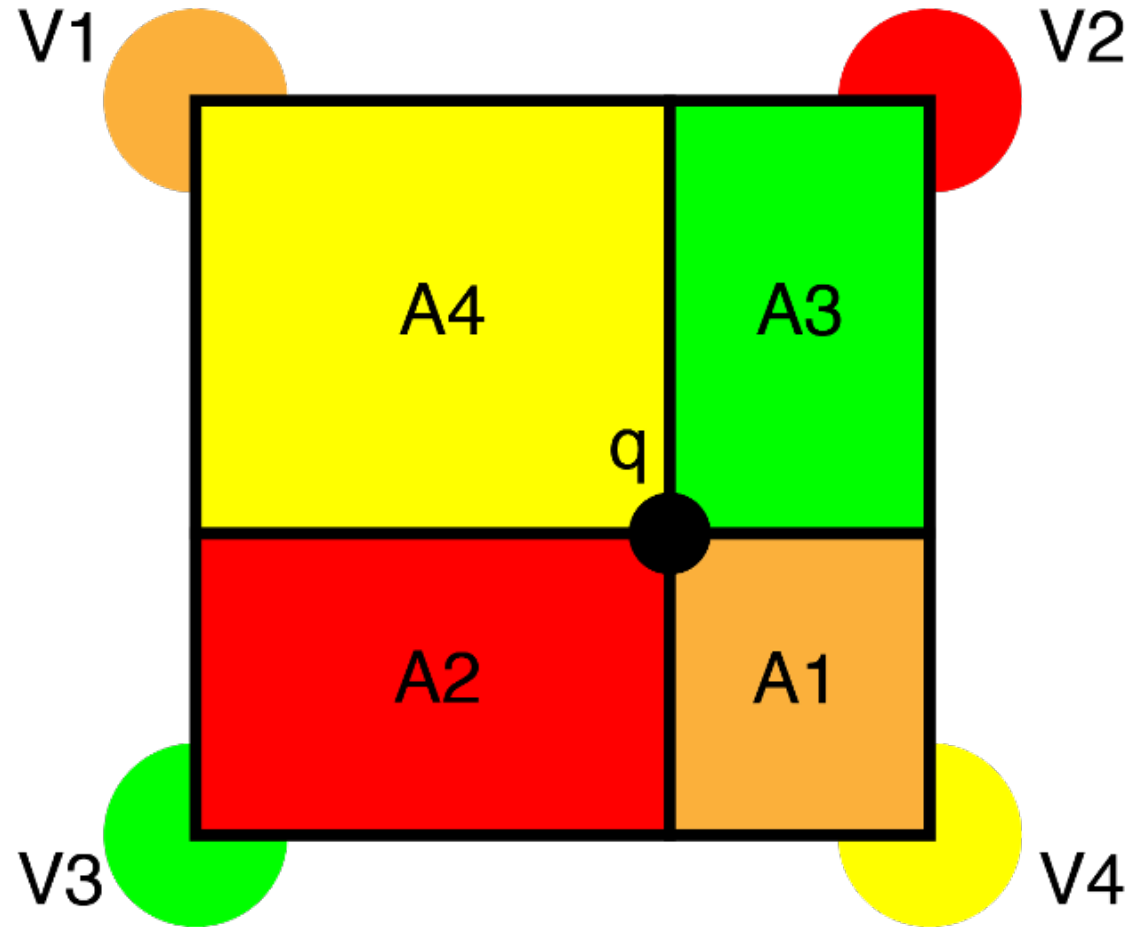
This time find the closest pixels
in a box

Same plan, weighted sum based
on area of opposite rectangle

$$Q = V1 * A1 + V2 * A2 + V3 * A3 + V4 * A4$$

Still need to normalize!

Or do we?



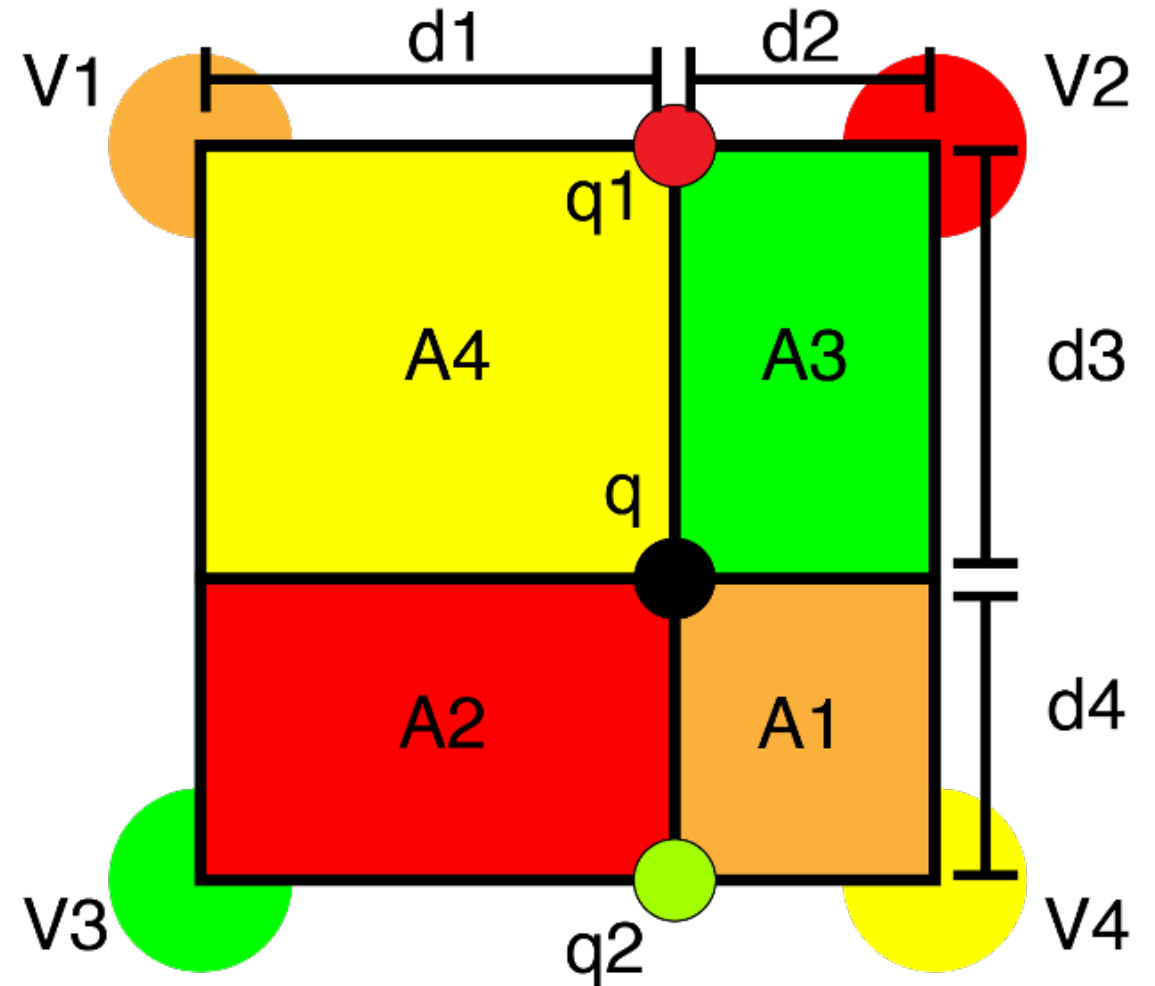
Bilinear interpolation: for grids, pretty good

Alternatively, linear interpolation
of linear interpolates

$$q1 = V1 * d2 + V2 * d1$$

$$q2 = V3 * d2 + V4 * d1$$

$$q = q1 * d4 + q2 * d3$$



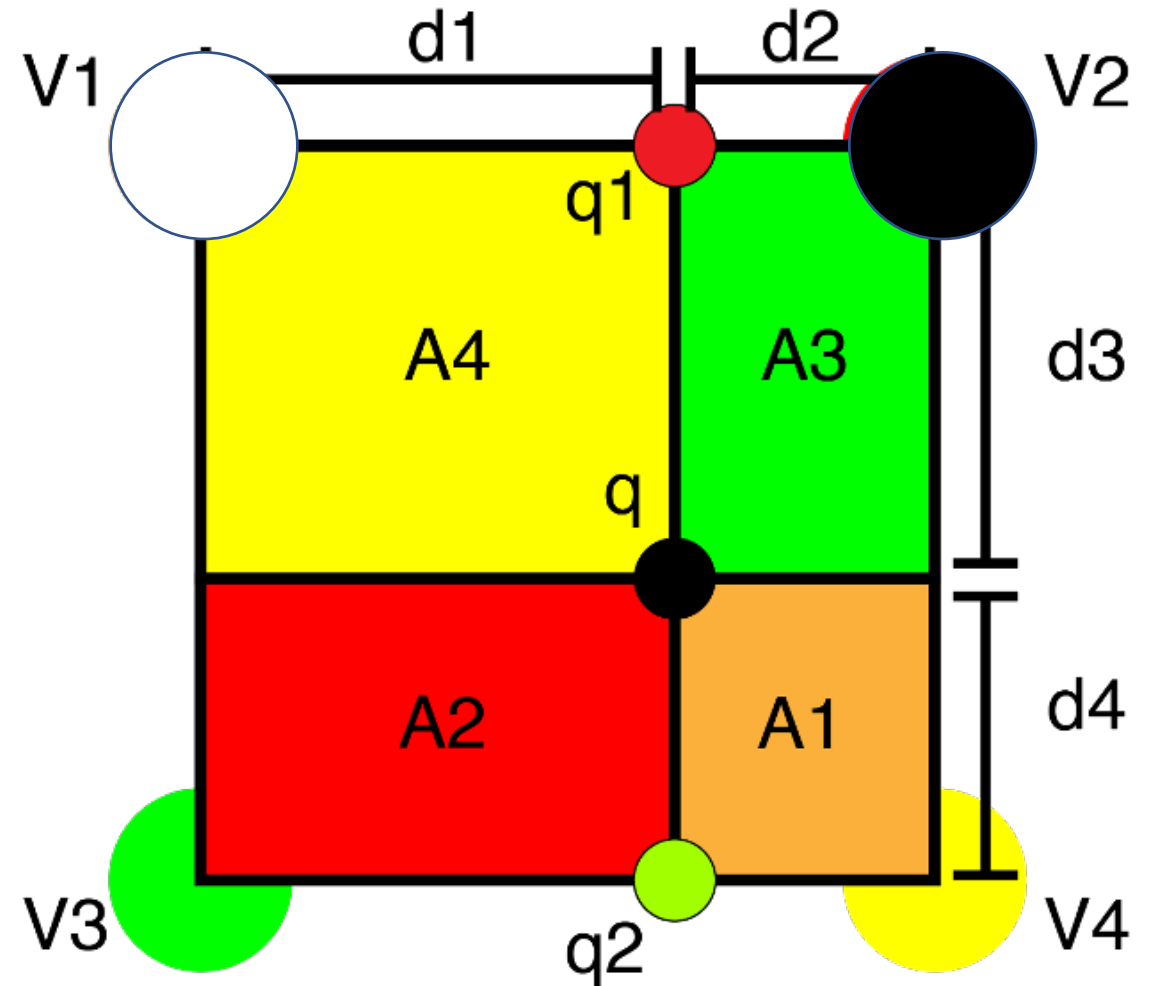
Bilinear interpolation: for grids, pretty good

Alternatively, linear interpolation
of linear interpolates

$$q1 = V1 * d2 + V2 * d1$$

$$q2 = V3 * d2 + V4 * d1$$

$$q = q1 * d4 + q2 * d3$$



Bilinear interpolation: for grids, pretty good

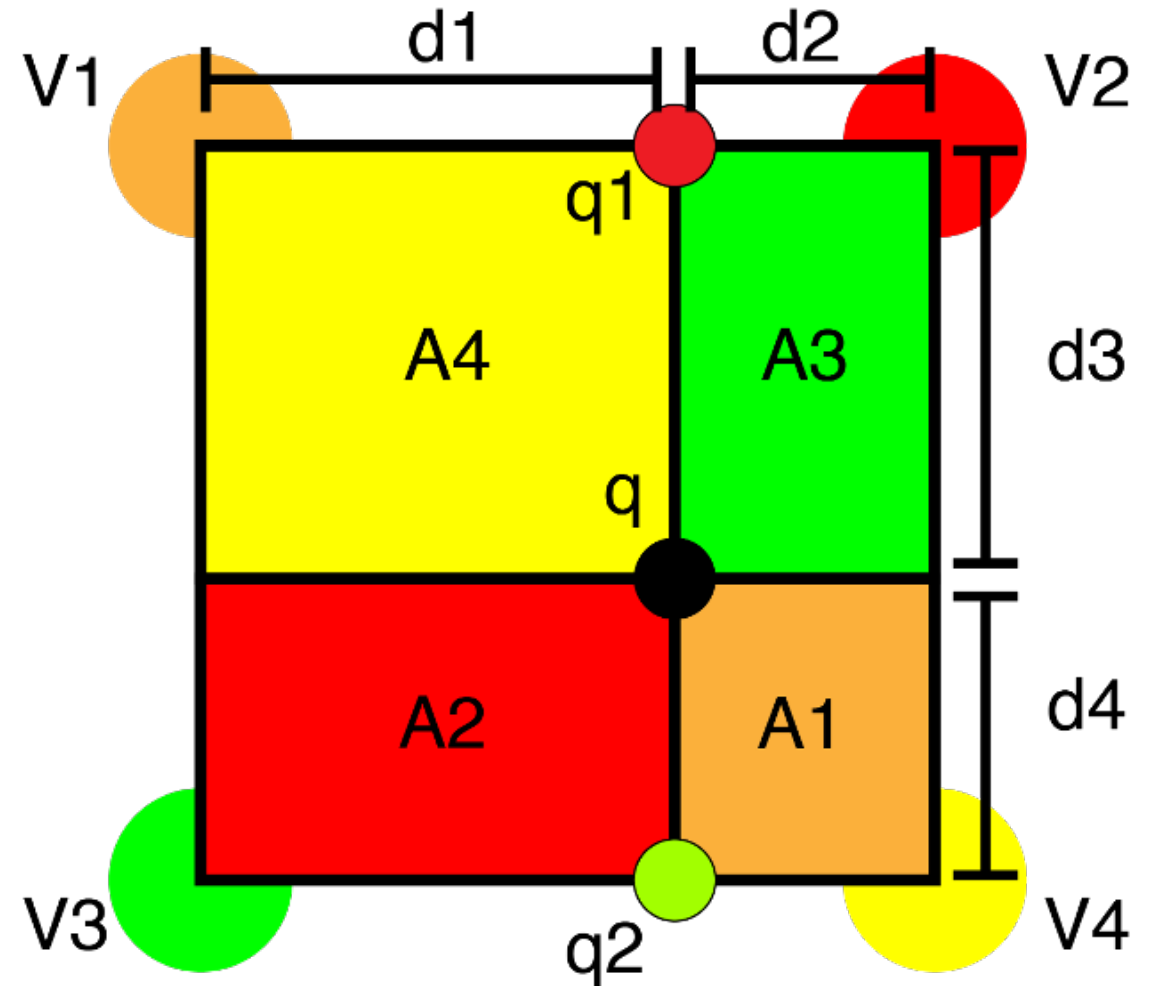
$$q1 = V1*d2 + V2*d1$$

$$q2 = V3*d2 + V4*d1$$

$$q = q1*d4 + q2*d3$$

Equivalent:

$$q = q1*d4 + q2*d3$$



Bilinear interpolation: for grids, pretty good

$$q1 = V1*d2 + V2*d1$$

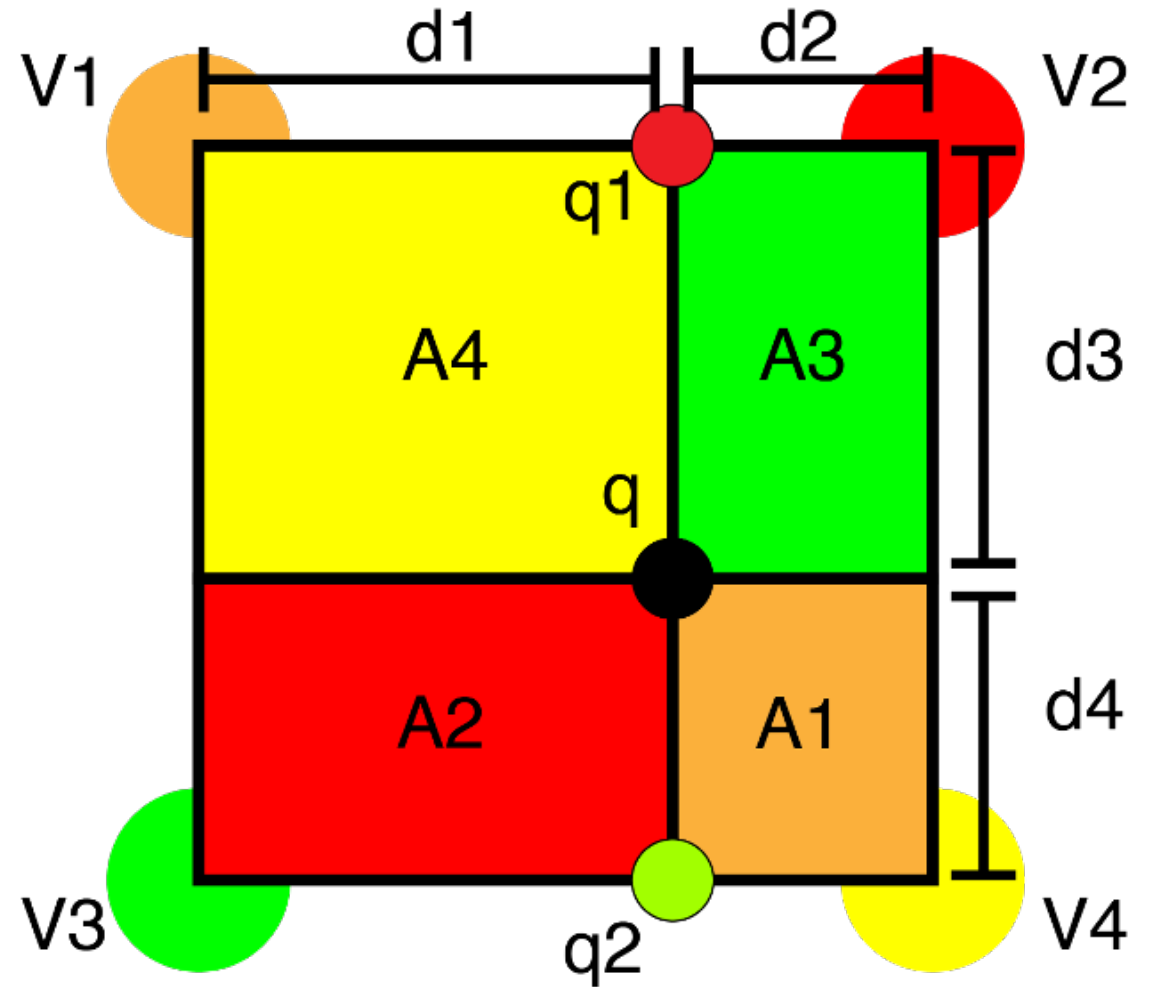
$$q2 = V3*d2 + V4*d1$$

$$q = q1*d4 + q2*d3$$

Equivalent:

$$q = q1*d4 + q2*d3$$

$$q = (V1*d2 + V2*d1)*d4 + (V3*d2 + V4*d1)*d3 \text{ (subst)}$$



Bilinear interpolation: for grids, pretty good

$$q1 = V1*d2 + V2*d1$$

$$q2 = V3*d2 + V4*d1$$

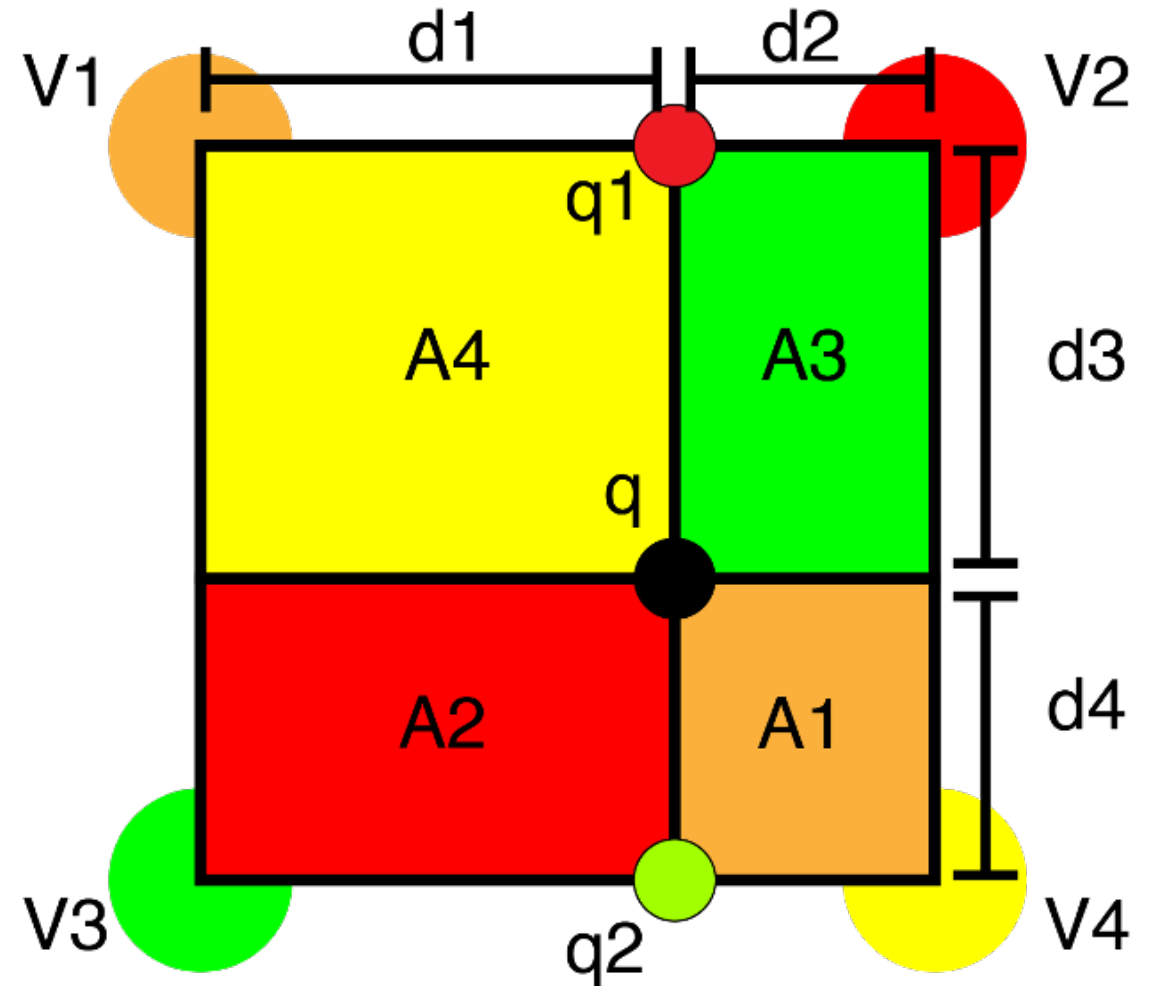
$$q = q1*d4 + q2*d3$$

Equivalent:

$$q = q1*d4 + q2*d3$$

$$q = (V1*d2 + V2*d1)*d4 + (V3*d2 + V4*d1)*d3 \text{ (subst)}$$

$$q = V1*d2*d4 + V2*d1*d4 + V3*d2*d3 + V4*d1*d3 \text{ (distribution)}$$



Bilinear interpolation: for grids, pretty good

$$q1 = V1*d2 + V2*d1$$

$$q2 = V3*d2 + V4*d1$$

$$q = q1*d4 + q2*d3$$

Equivalent:

$$q = q1*d4 + q2*d3$$

$$q = (V1*d2 + V2*d1)*d4 + (V3*d2 + V4*d1)*d3 \text{ (subst)}$$

$$q = V1*d2*d4 + V2*d1*d4 + V3*d2*d3 + V4*d1*d3 \text{ (distribution)}$$

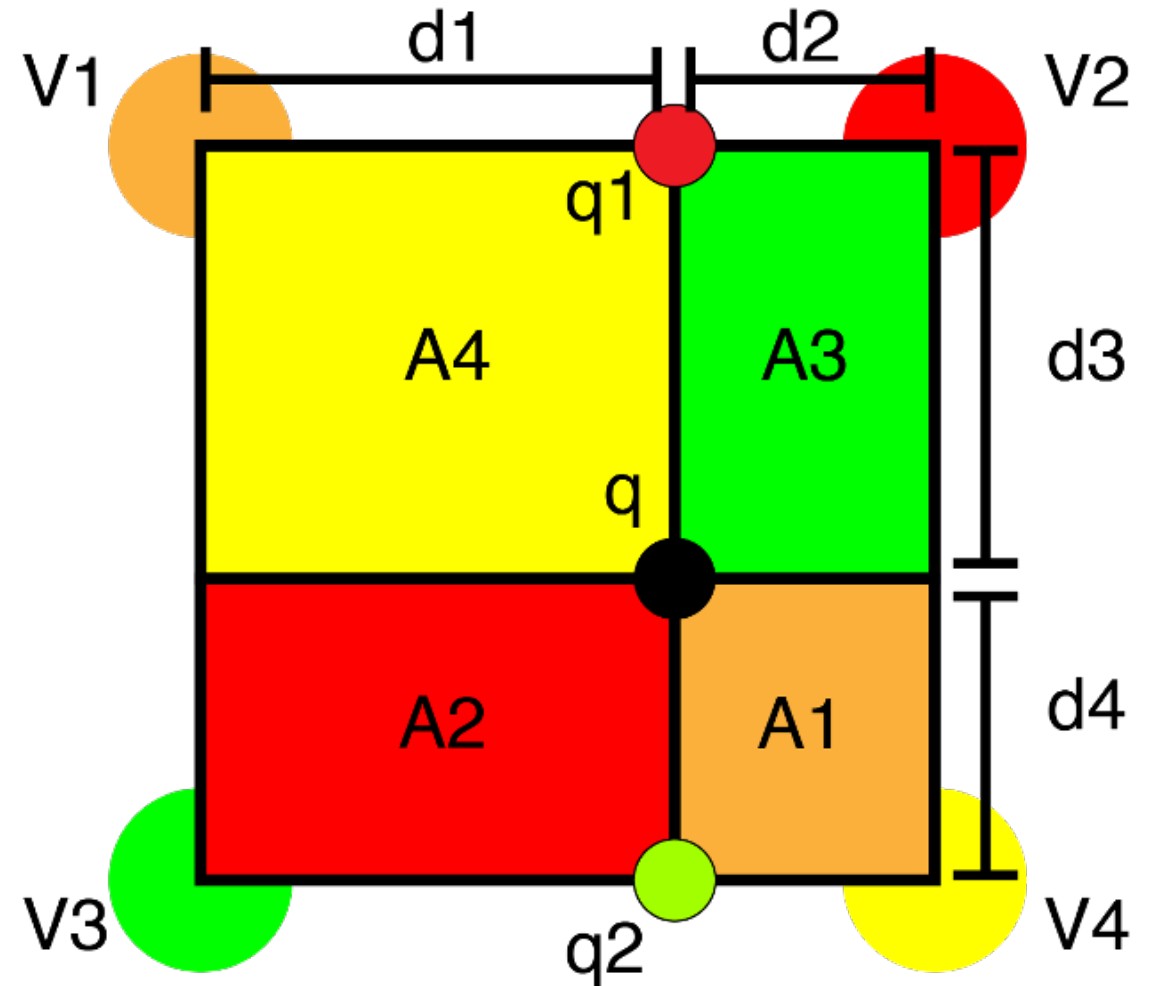
Recall:

$$A1 = d2*d4$$

$$A2 = d1*d4$$

$$A3 = d2*d3$$

$$A4 = d1*d3$$



Bilinear interpolation: for grids, pretty good

$$q1 = V1*d2 + V2*d1$$

$$q2 = V3*d2 + V4*d1$$

$$q = q1*d4 + q2*d3$$

Equivalent:

$$q = q1*d4 + q2*d3$$

$$q = (V1*d2 + V2*d1)*d4 + (V3*d2 + V4*d1)*d3 \text{ (subst)}$$

$$q = V1*d2*d4 + V2*d1*d4 + V3*d2*d3 + V4*d1*d3 \text{ (distribution)}$$

Recall:

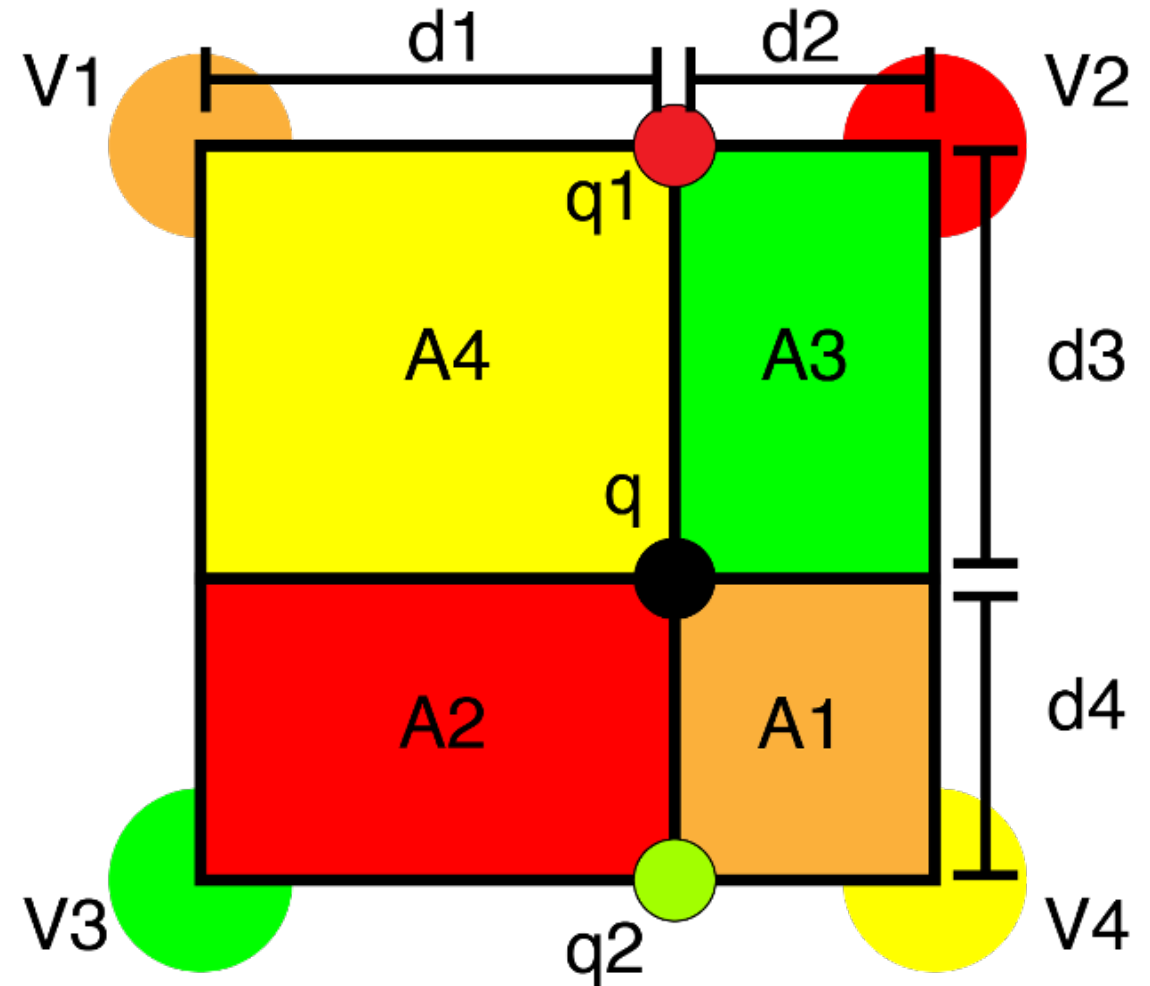
$$A1 = d2*d4$$

$$A2 = d1*d4$$

$$A3 = d2*d3$$

$$A4 = d1*d3$$

$$q = V1*A1 + V2*A2 + V3*A3 + V4*A4$$



Bilinear interpolation: for grids, pretty good

$$q1 = V1*d2 + V2*d1$$

$$q2 = V3*d2 + V4*d1$$

$$q = q1*d4 + q2*d3$$

Equivalent:

$$q = q1*d4 + q2*d3$$

$$q = (V1*d2 + V2*d1)*d4 + (V3*d2 + V4*d1)*d3 \text{ (subst)}$$

$$q = V1*d2*d4 + V2*d1*d4 + V3*d2*d3 + V4*d1*d3 \text{ (distribution)}$$

Recall:

$$A1 = d2*d4$$

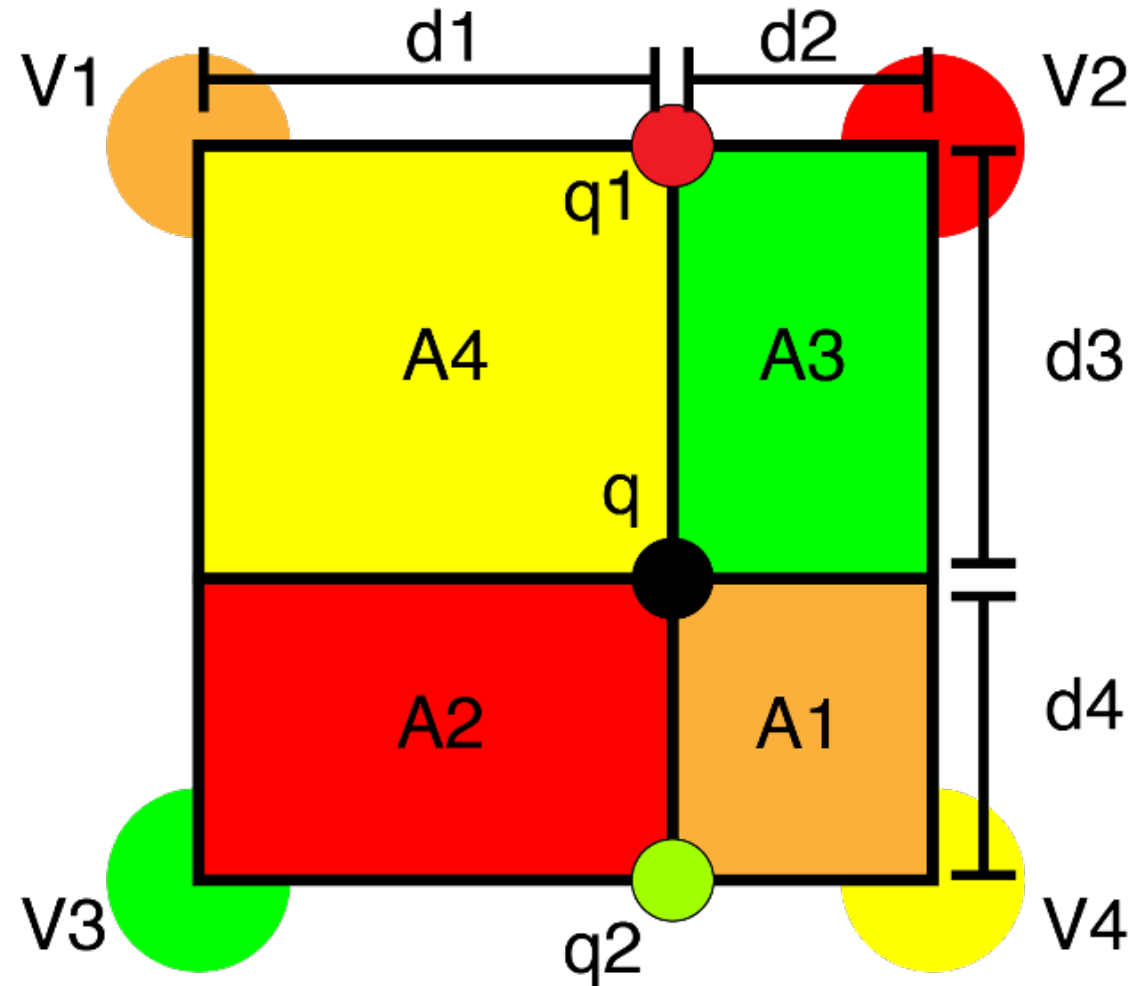
$$A2 = d1*d4$$

$$A3 = d2*d3$$

$$A4 = d1*d3$$

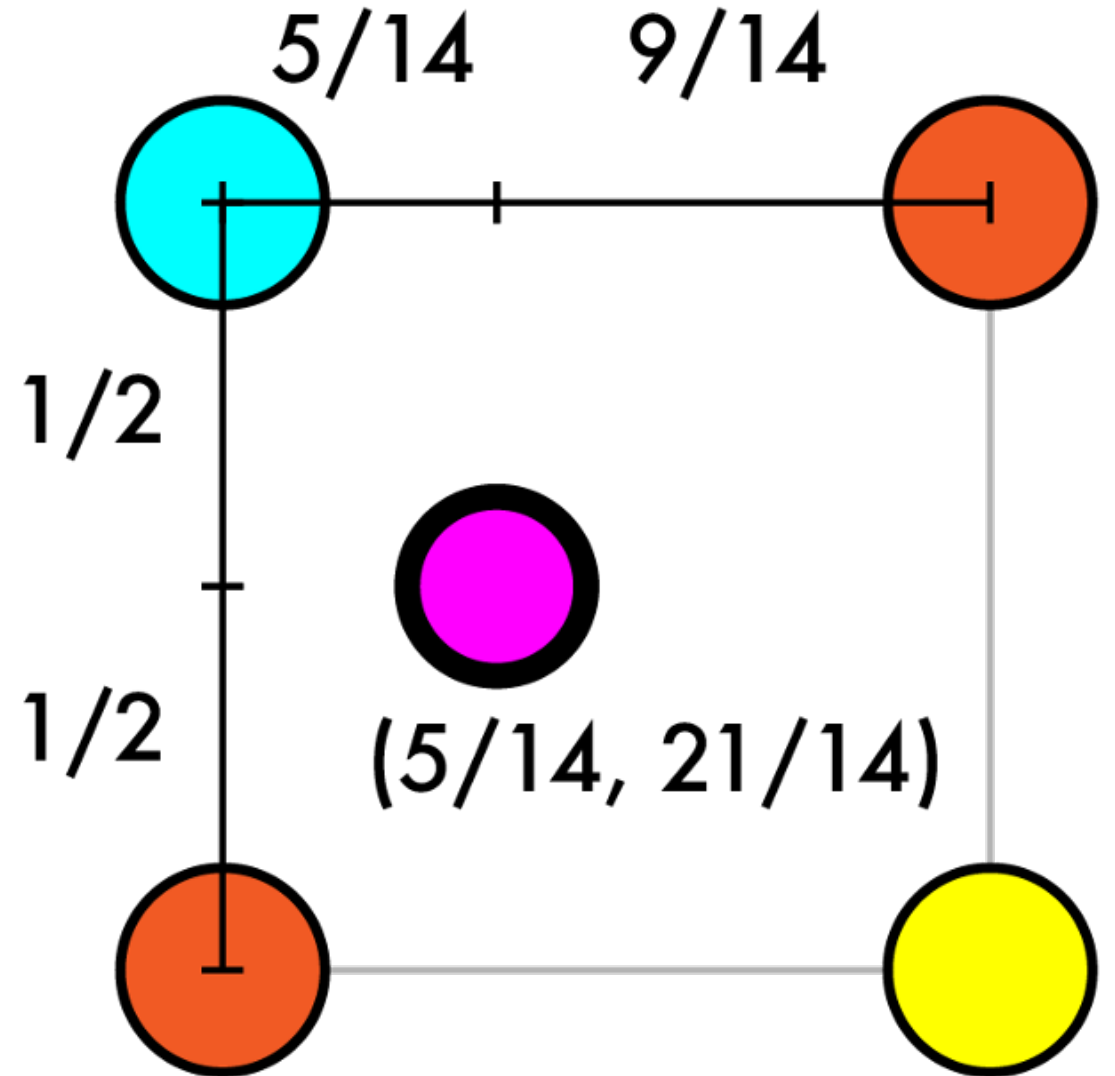
$$q = V1*A1 + V2*A2 + V3*A3 + V4*A4$$

yay!



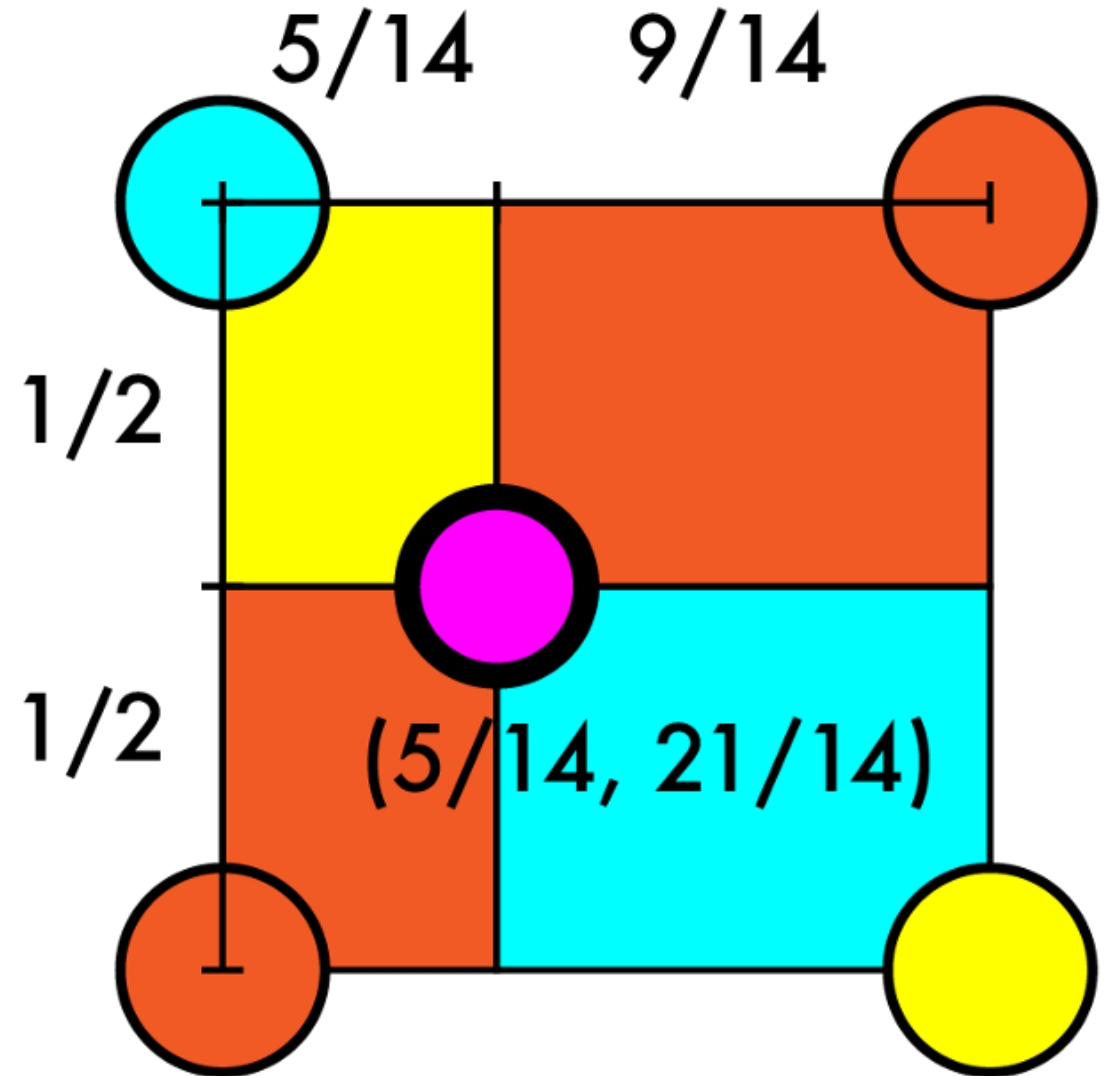
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values



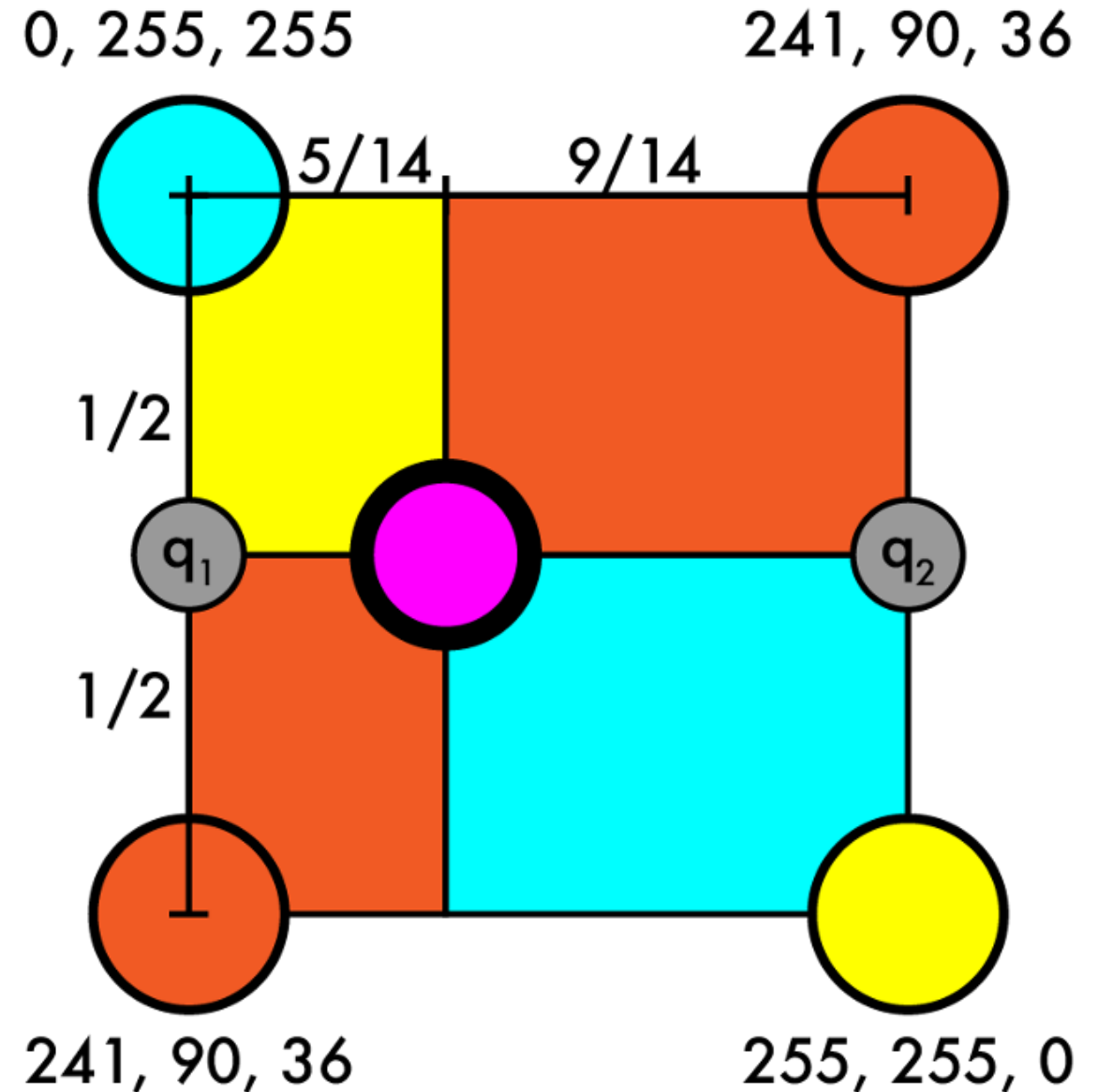
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - Size of opposite rects



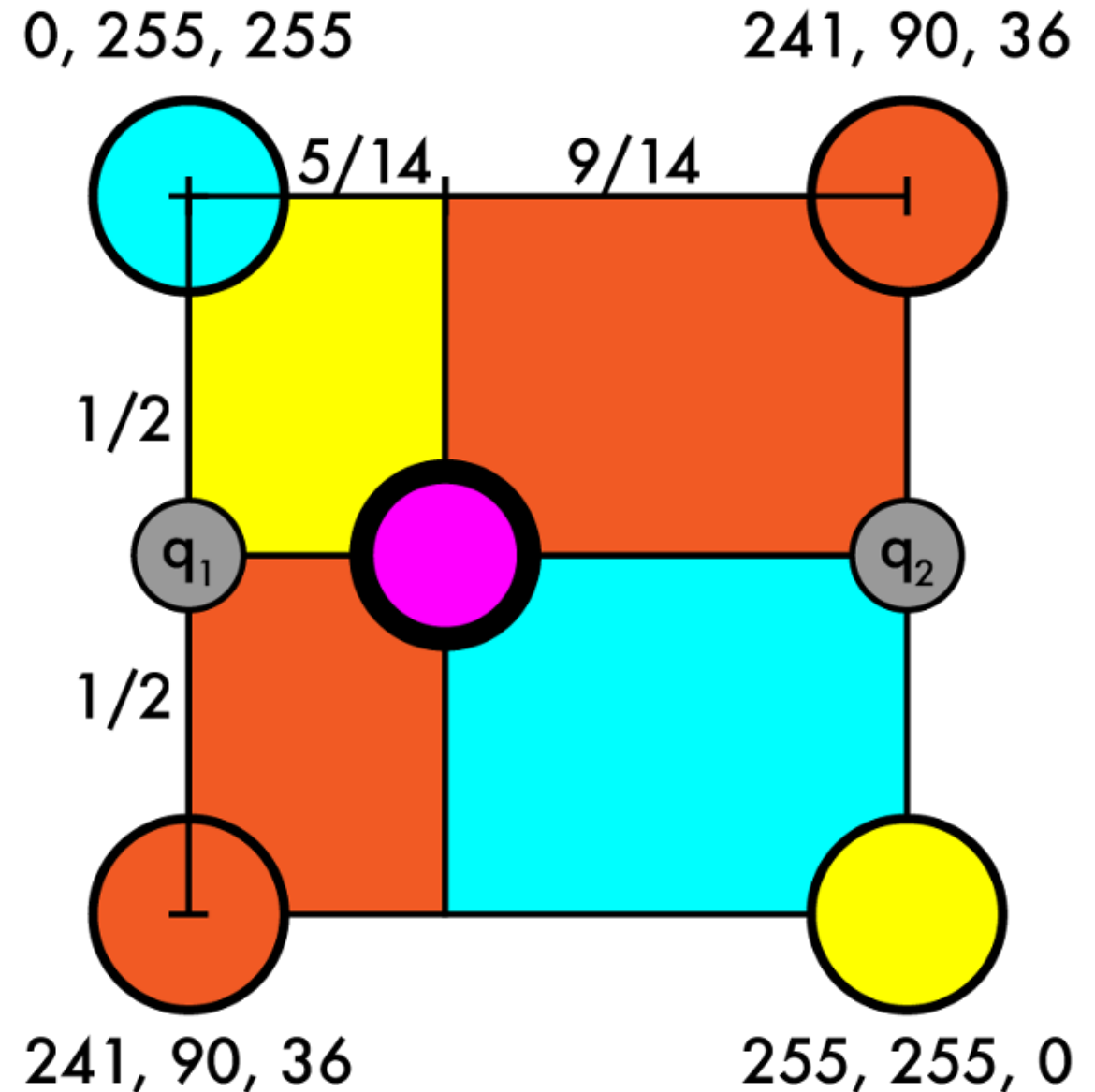
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - Size of opposite rects
 - OR find q_1 and q_2 , then interpolate between them



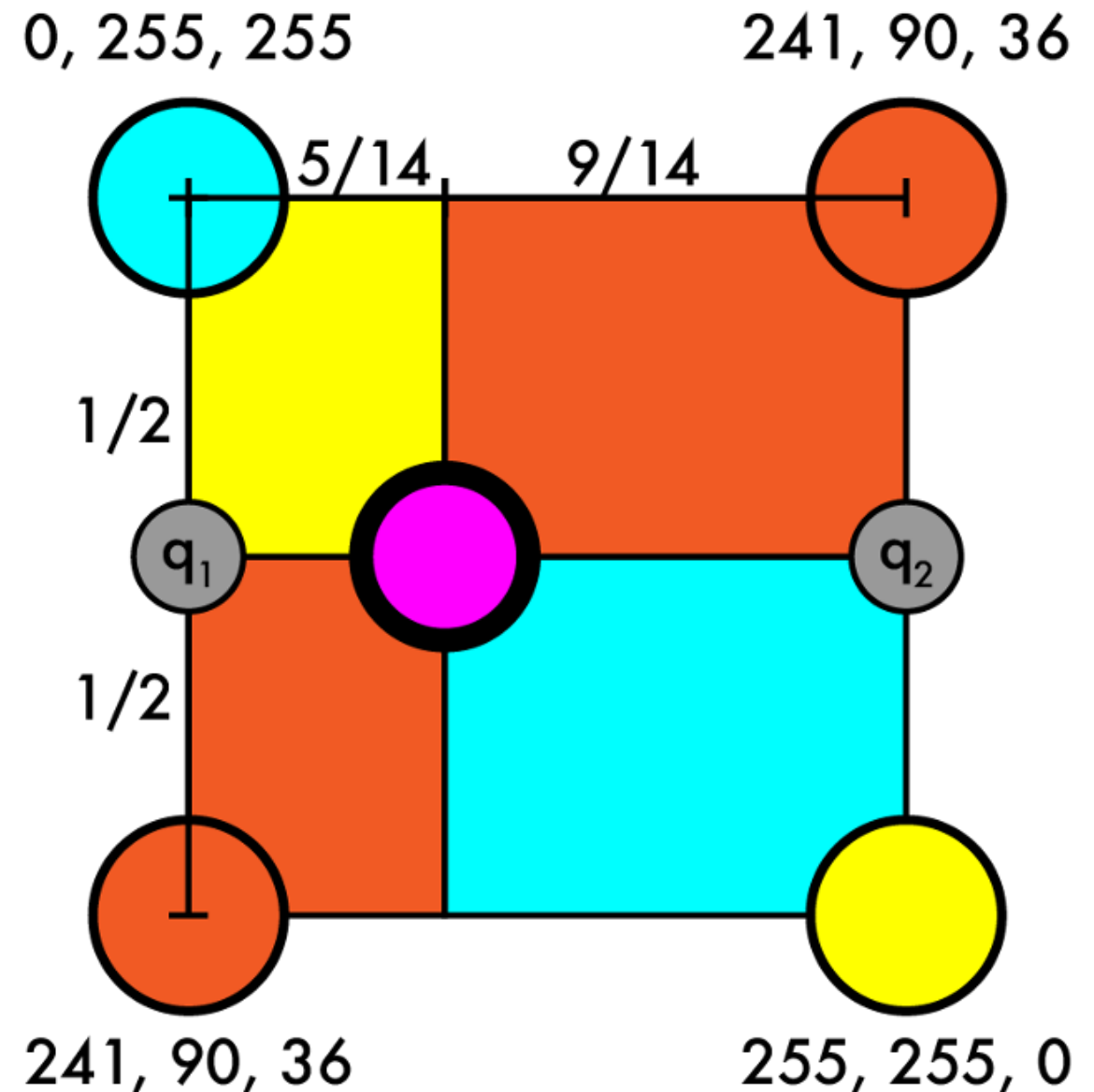
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q1 = r1, g1, b1$
 - $r1 = .5 * 0 + .5 * 241$
 - $g1 = .5 * 255 + .5 * 90$
 - $b1 = .5 * 241 + .5 * 36$



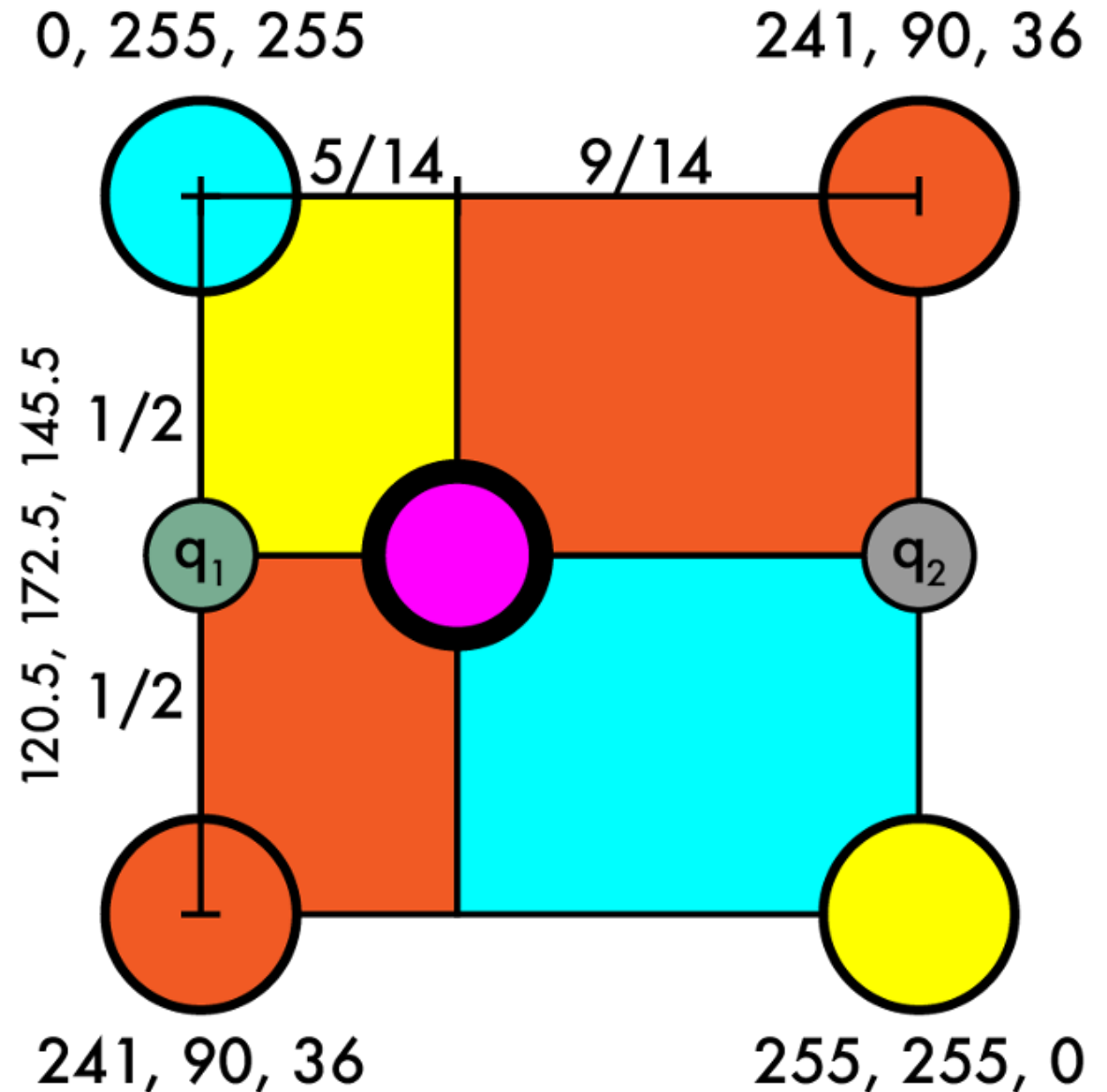
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q1 = r1, g1, b1$
 - $r1 = .5 * 0 + .5 * 241$
 - $g1 = .5 * 255 + .5 * 90$
 - $b1 = .5 * 241 + .5 * 36$



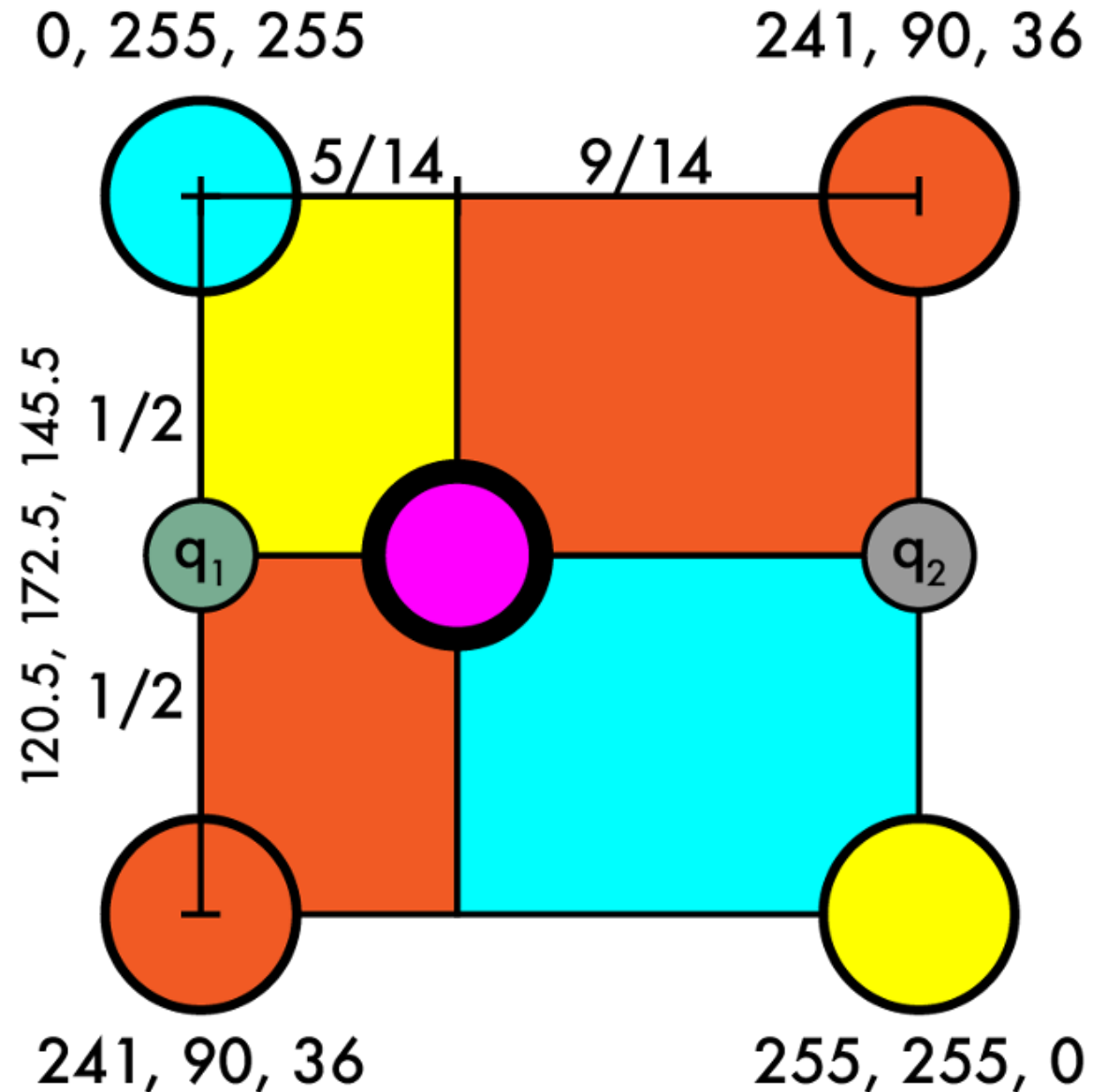
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q_1 = (120.5, 172.5, 145.5)$
 - $q_2 = r_2, g_2, b_2$



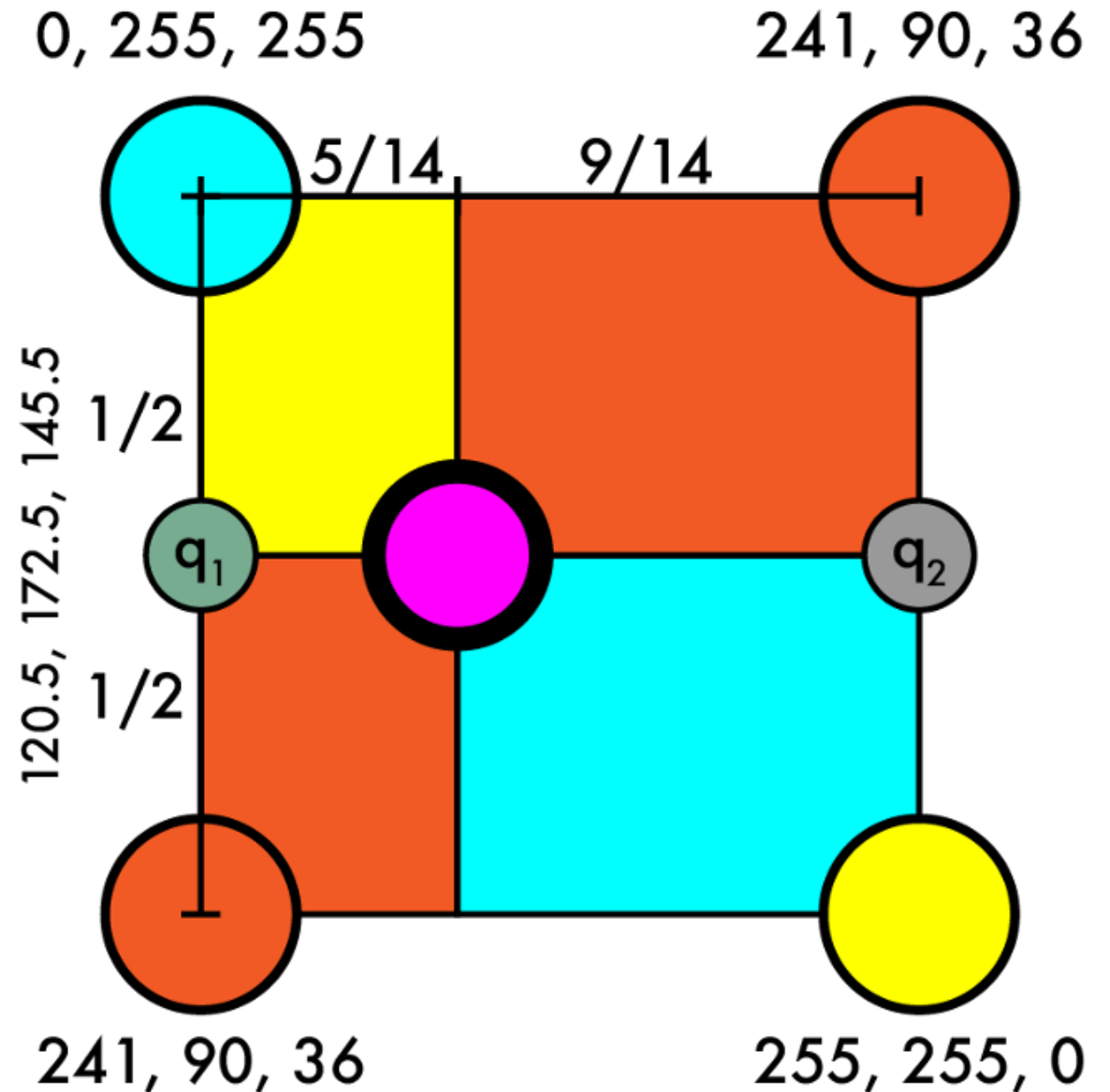
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q1 = (120.5, 172.5, 145.5)$
 - $q2 = r2, g2, b2$



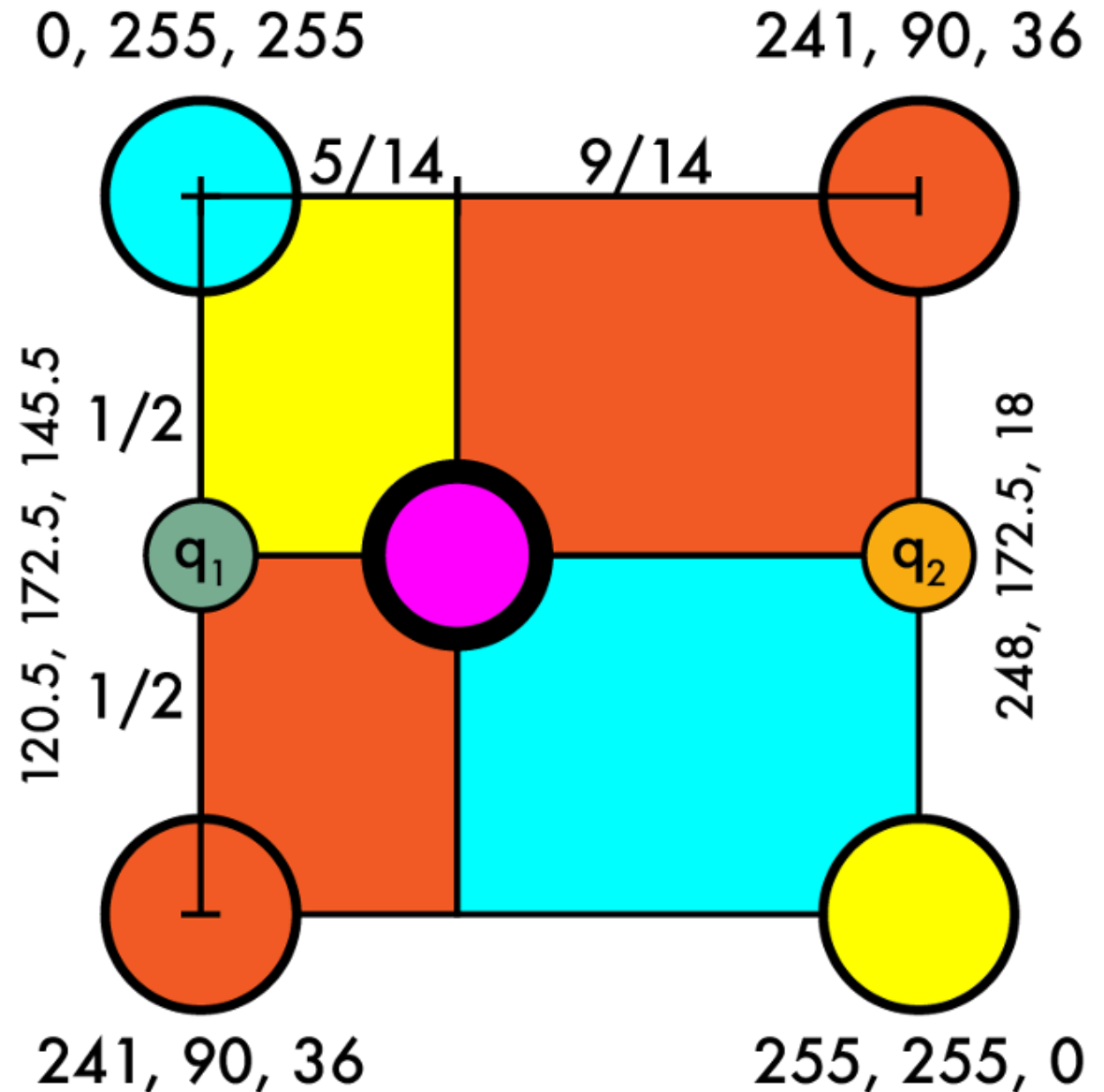
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q1 = (120.5, 172.5, 145.5)$
 - $q2 = r2, g2, b2$
 - $r2 = .5 * 241 + .5 * 255$
 - $g2 = .5 * 90 + .5 * 255$
 - $b2 = .5 * 36 + .5 * 0$



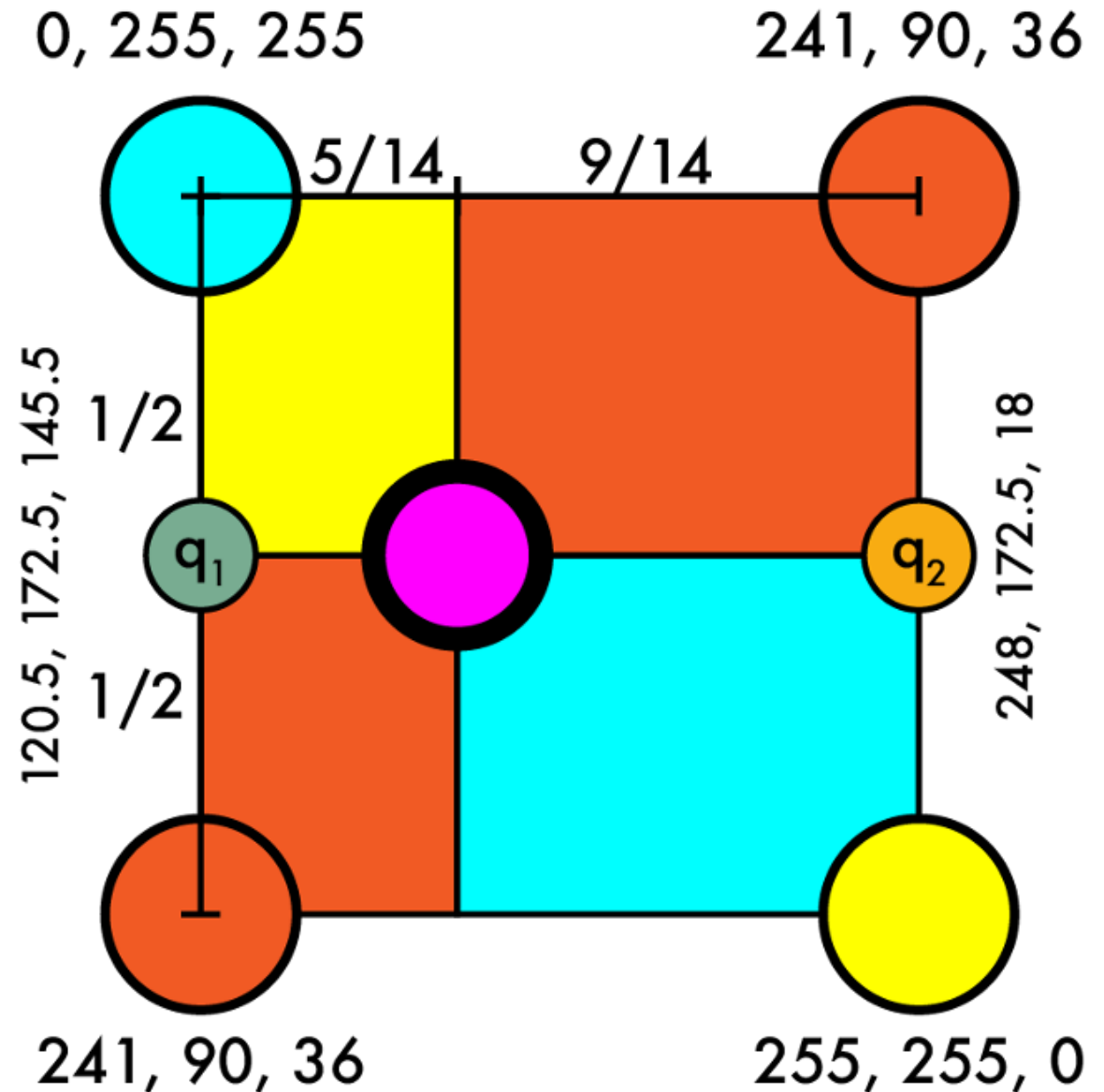
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q_1 = (120.5, 172.5, 145.5)$
 - $q_2 = (248, 172.5, 18)$



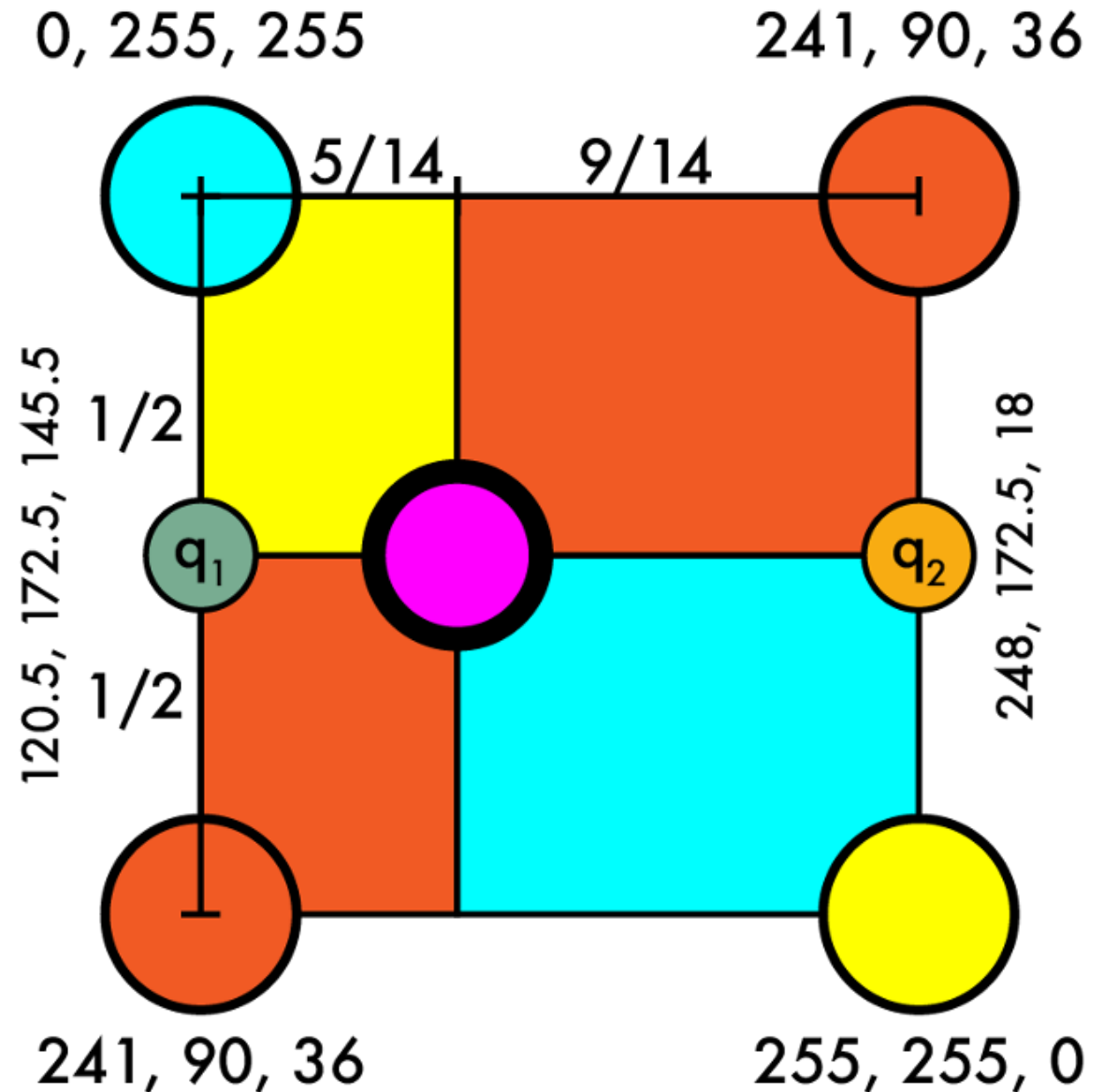
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q_1 = (120.5, 172.5, 145.5)$
 - $q_2 = (248, 172.5, 18)$
 - $q = r, g, b$
 - $w = 9/14 * w_1 + 5/14 w_2$



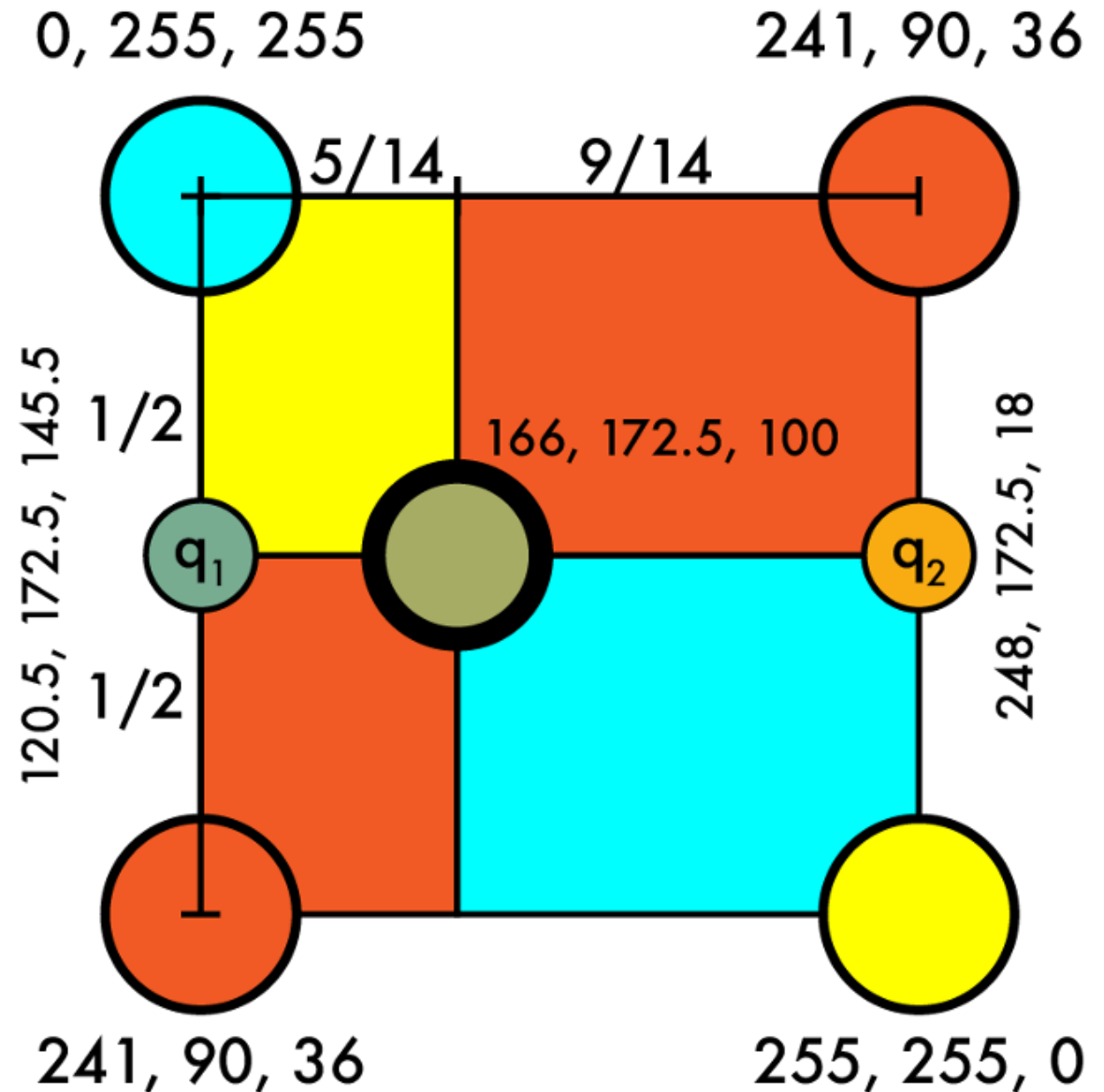
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - (1, 3) -> (5/14, 7/14)
 - Interpolate old values
 - $q_1 = (120.5, 172.5, 145.5)$
 - $q_2 = (248, 172.5, 18)$
 - $q = r, g, b$
 - $w = 9/14 * w_1 + 5/14 w_2$
 - $r = 9/14 * 120.5 + 5/14 * 248$
 - $g = 9/14 * 172.5 + 5/14 * 172.5$
 - $b = 9/14 * 145.5 + 5/14 * 18$



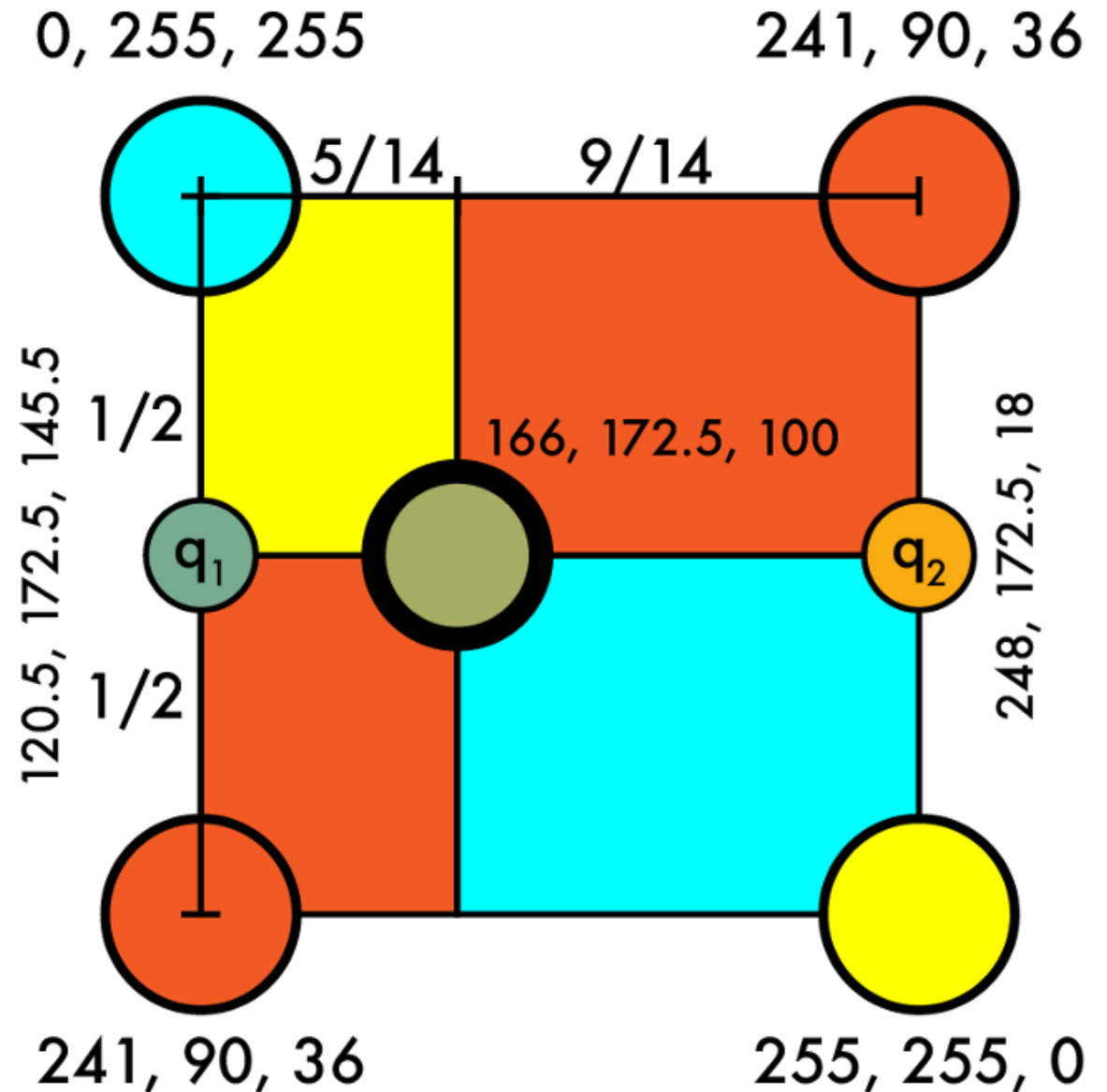
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q_1 = (120.5, 172.5, 145.5)$
 - $q_2 = (248, 172.5, 18)$
 - $q = (166, 172.5, 100)$



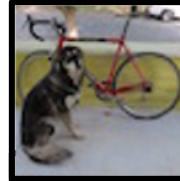
Resize 4x4 -> 7x7

- Create our new image
- Match up coordinates
 - $4/7 X - 3/14 = Y$
- Iterate over new pts
 - Map to old coords
 - $(1, 3) \rightarrow (5/14, 7/14)$
 - Interpolate old values
 - $q = (166, 172.5, 100)$



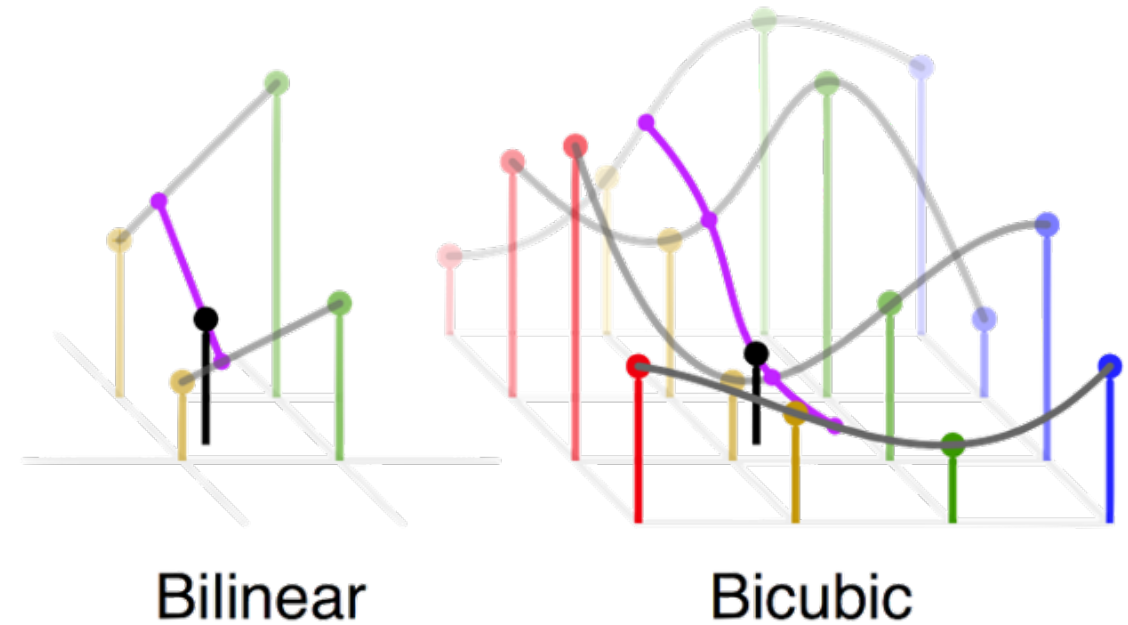
Bilinear interpolation: for grids, pretty good

- Smoother than NN
- More complex
 - 4 lookups
 - Some math
- Often the right tradeoff of speed vs final result



Bicubic sampling: more complex, maybe better?

- A cubic interpolation of 4 cubic interpolations
- Smoother than bilinear, no “star”
- 16 nearest neighbors
- Fit 3rd order poly:
 - $f(x) = a + bx + cx^2 + dx^3$
- Interpolate along axis
- Fit another poly to interpolated values



Bicubic vs bilinear



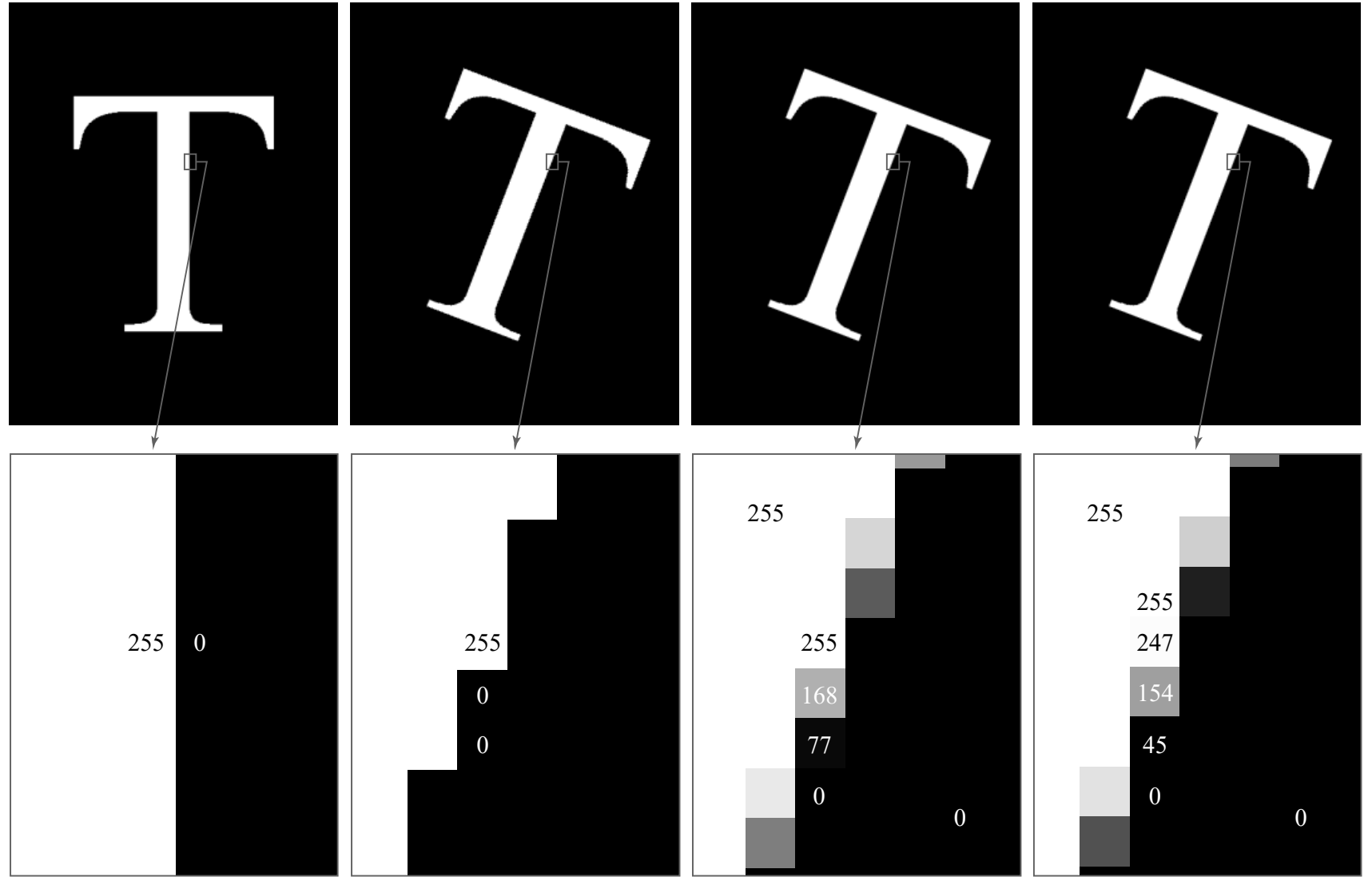


Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Adaptive threshold	.5
Image Histogram	.6
Otus method	.7
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

cv2.adaptiveThreshold

1.src: The source image,.

2.maxValue: The maximum value to use with the THRESH_BINARY and THRESH_BINARY_INV thresholding types..

3.adaptiveMethod: :

1. cv2.ADAPTIVE_THRESH_MEAN_C: The threshold value is the mean of the neighborhood area minus the constant C.
2. cv2.ADAPTIVE_THRESH_GAUSSIAN_C: The threshold value is a weighted sum of the neighborhood values where weights are a Gaussian window, minus the constant C.

4.thresholdType: cv2.THRESH_BINARY or cv2.THRESH_BINARY_INV.

5.blockSize: It determines the size of the neighborhood

6.C: A constant that is subtracted from the mean

```
img = cv.imread('sudoku.png',0)
```

```
img = cv.medianBlur(img,5)
```

```
ret,th1 =
```

```
cv.threshold(img,127,255,cv.THRESH_
```

```
BINARY)
```

```
th2 =
```

```
cv.adaptiveThreshold(img,255,cv.ADA
```

```
PTIVE_THRESH_MEAN_C,\
```

```
cv.THRESH_BINARY,11,2)
```

```
th3 =
```

```
cv.adaptiveThreshold(img,255,cv.ADA
```

```
PTIVE_THRESH_GAUSSIAN_C,\
```

```
cv.THRESH_BINARY,11,2)
```

Global Thresholding (v = 127)



Adaptive Gaussian Thresholding



Original Image



Adaptive Mean Thresholding



```

img = cv.imread('sudoku.png',0)
img = cv.medianBlur(img,5)
ret,th1 =
cv.threshold(img,127,255,cv.THRESH_
BINARY)

th2 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_MEAN_C,\
cv.THRESH_BINARY,11,2)

th3 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_GAUSSIAN_C,\
cv.THRESH_BINARY,11,2)

```



```

img = cv.imread('sudoku.png',0)
img = cv.medianBlur(img,5)
ret,th1 =
cv.threshold(img,127,255,cv.THRESH_
BINARY)

th2 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_MEAN_C,\
cv.THRESH_BINARY,11,2)

th3 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_GAUSSIAN_C,\
cv.THRESH_BINARY,11,2)

```

Global Thresholding (v = 127)



Adaptive Gaussian Thresholding



Original Image



Adaptive Mean Thresholding




```

img = cv.imread('sudoku.png',0)
img = cv.medianBlur(img,5)
ret,th1 =
cv.threshold(img,127,255,cv.THRESH_
BINARY)

th2 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_MEAN_C,\
cv.THRESH_BINARY,11,2)

th3 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_GAUSSIAN_C,\
cv.THRESH_BINARY,11,2)

```

Global Thresholding (v = 127)



Adaptive Gaussian Thresholding



Original Image



Adaptive Mean Thresholding



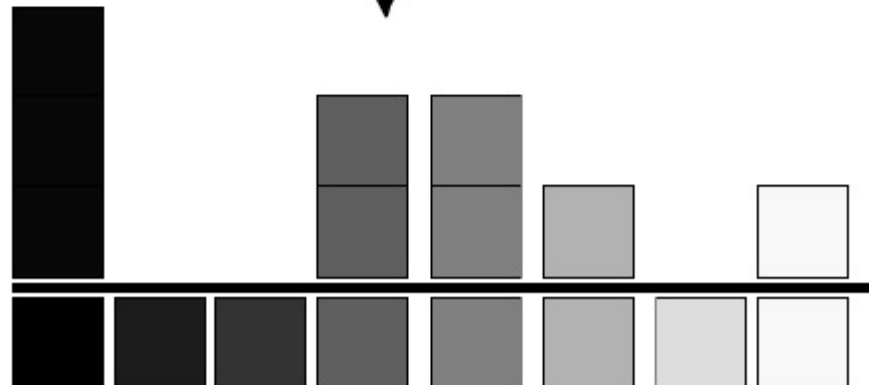
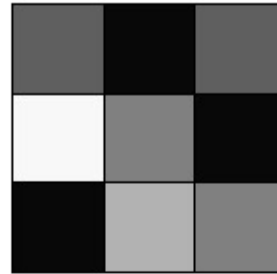
Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Adaptive threshold	.5
Image Histogram	.6
Otus method	.7
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

Image Histogram

מה זה היסטוגרמה של תמונה ?

נתונה תמונה עם 9 פיקסלים.



ההיסטוגרמה:
(כמה פיקסלים מכל צבע\גוון)

Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Adaptive threshold	.5
Image Histogram	.6
Otus method	.7
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

Otsu's method

Binarization divides the histogram of the images in two groups, namely class 0 and class 1, as illustrated [Fig. 48](#).

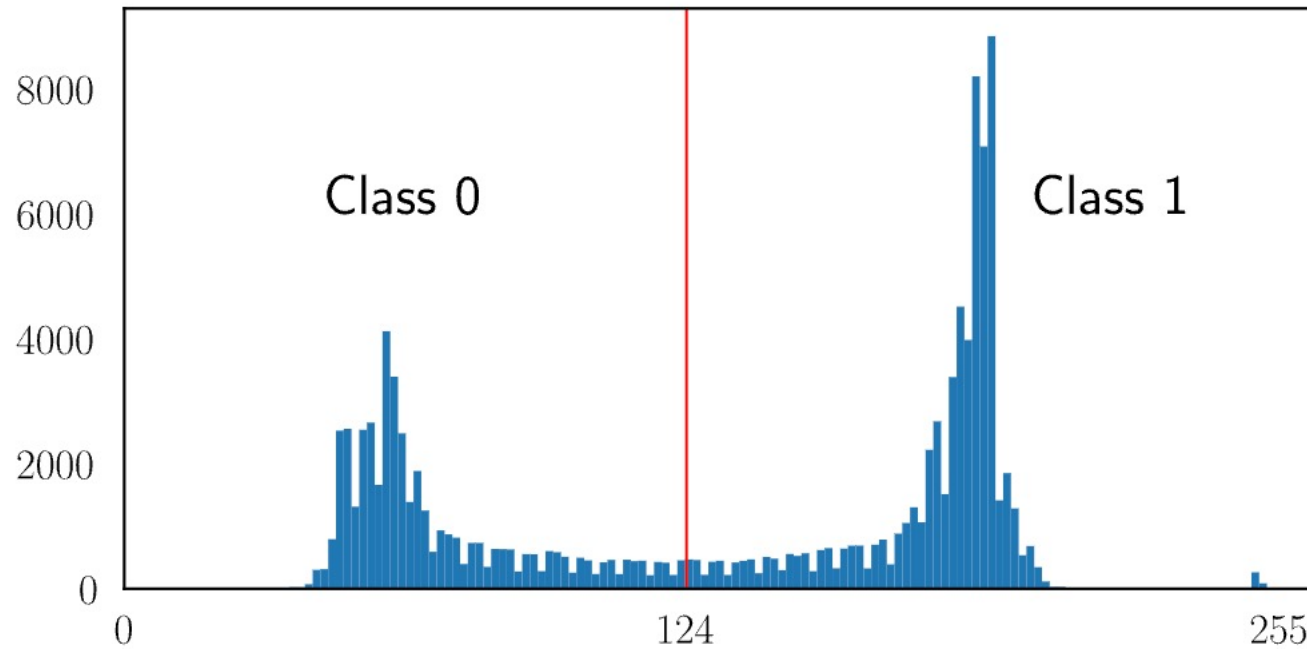
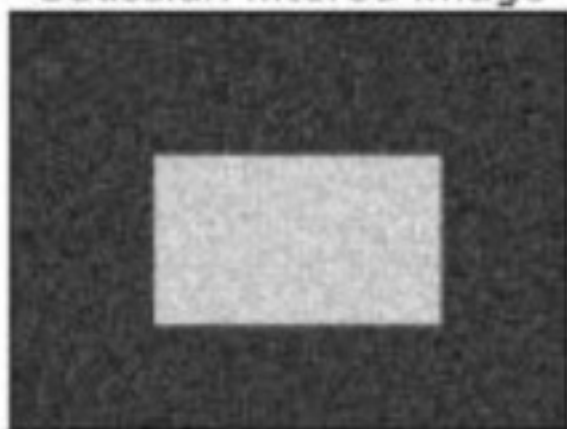
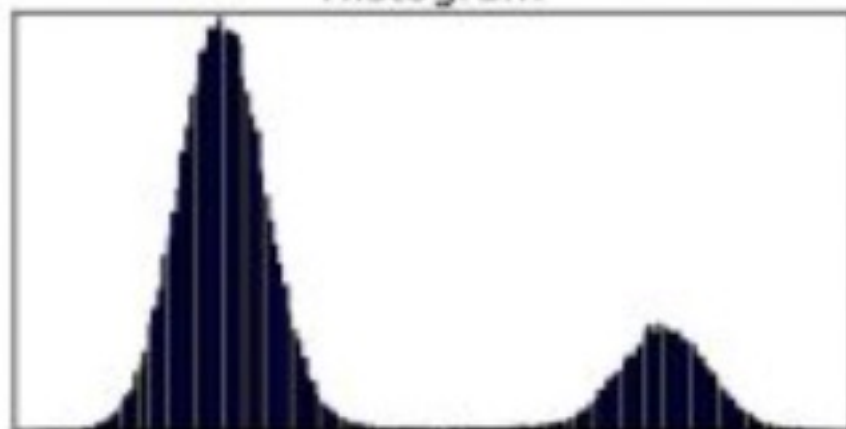


Fig. 48 An histogram and a threshold T .

Gaussian filtered Image



Histogram



Otsu's Thresholding

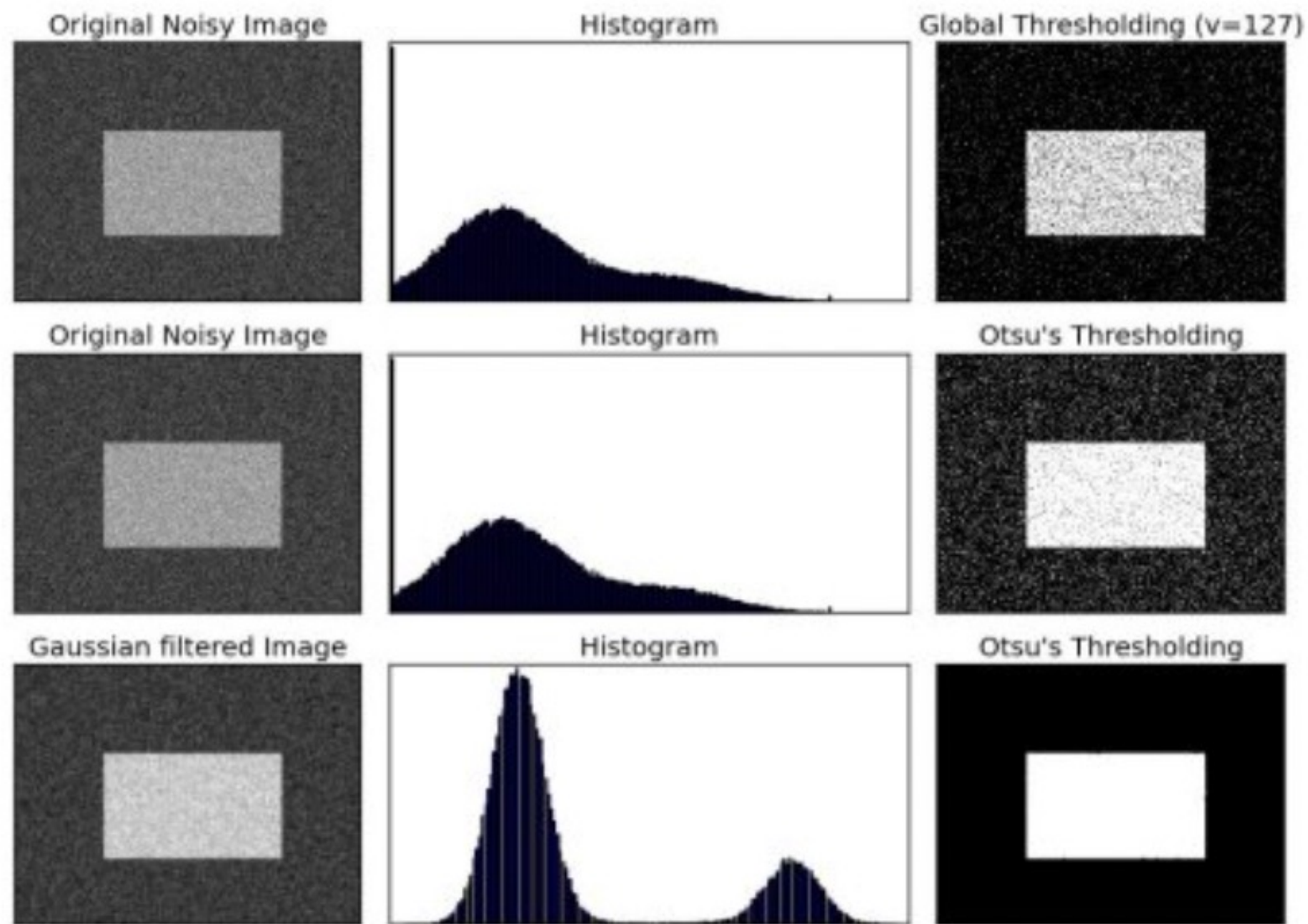


image

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt
img = cv.imread('noisy2.png', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
# global thresholding
ret1,th1 = cv.threshold(img,127,255,cv.THRESH_BINARY)
# Otsu's thresholding
ret2,th2 = cv.threshold(img,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
# Otsu's thresholding after Gaussian filtering
blur = cv.GaussianBlur(img,(5,5),0)
ret3,th3 = cv.threshold(blur,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
```

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt
img = cv.imread('noisy2.png', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists"
# global thresholding
ret1,th1 = cv.threshold(img,127,255,cv.THRESH_BINARY)
# Otsu's thresholding
ret2,th2 = cv.threshold(img,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
# Otsu's thresholding after Gaussian filtering
blur = cv.GaussianBlur(img,(5,5),0)
ret3,th3 = cv.threshold(blur,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)

for i in range(3):
    plt.subplot(3,3,i*3+1),plt.imshow(images[i*3],'gray')
    plt.title(titles[i*3]), plt.xticks([], plt.yticks([]))
    plt.subplot(3,3,i*3+2),plt.hist(images[i*3].ravel(),256)
    plt.title(titles[i*3+1]), plt.xticks([], plt.yticks([]))
    plt.subplot(3,3,i*3+3),plt.imshow(images[i*3+2],'gray')
    plt.title(titles[i*3+2]), plt.xticks([], plt.yticks([]))
plt.show()
```

image

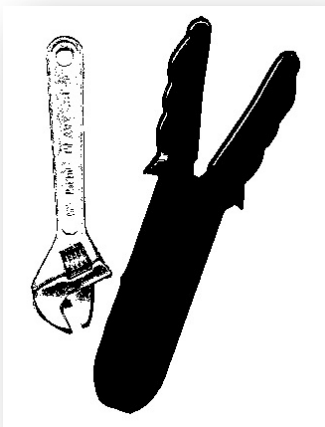
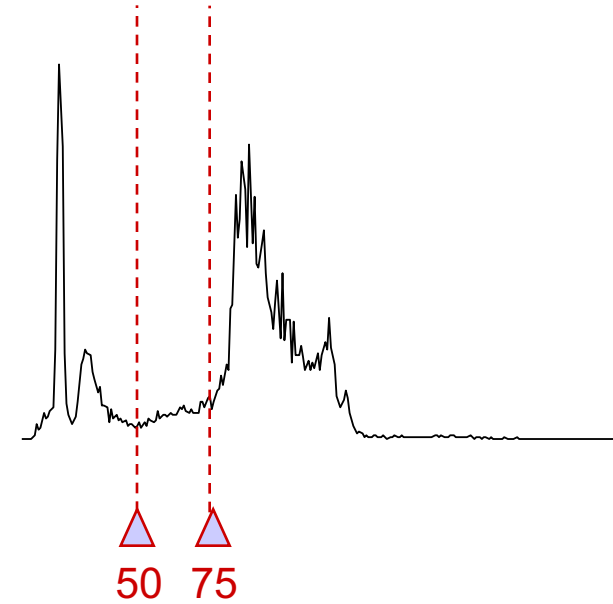
Always working ?

Segmentation using Thresholding

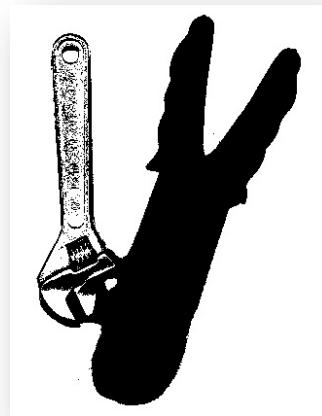
Original



Histogram



Threshold = 50



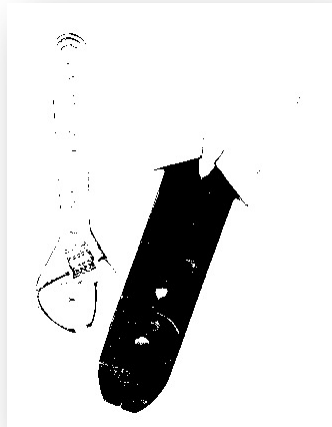
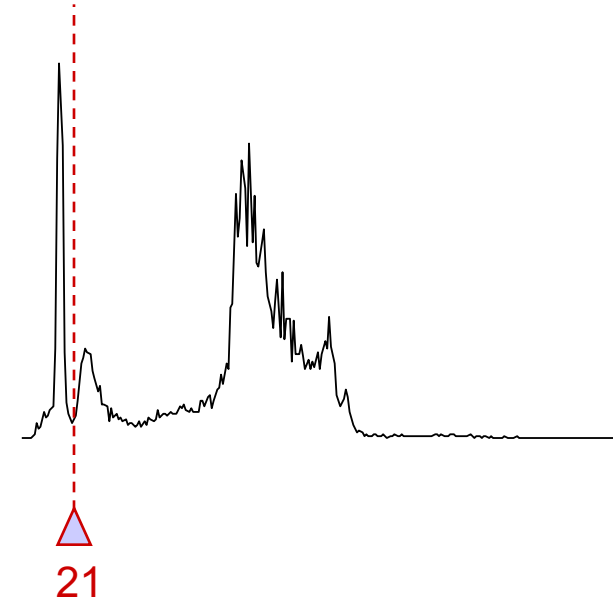
Threshold = 75

Segmentation using Thresholding

Original



Histogram



Threshold = 21

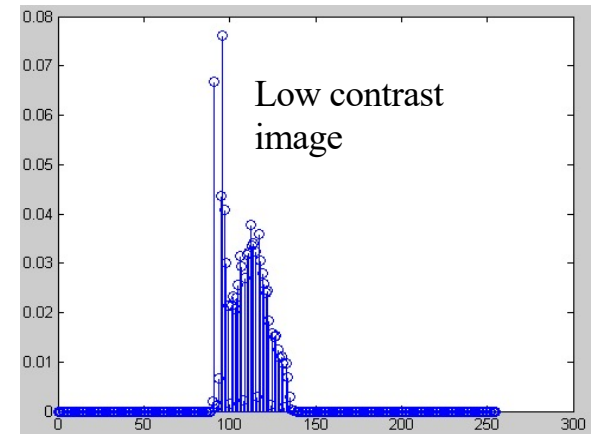
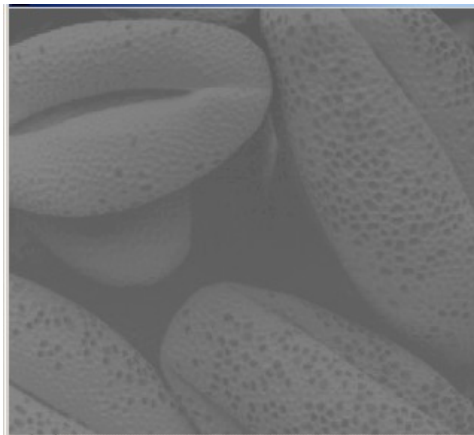
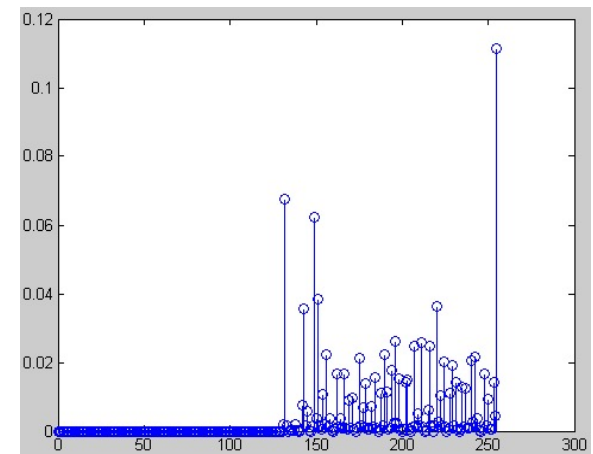
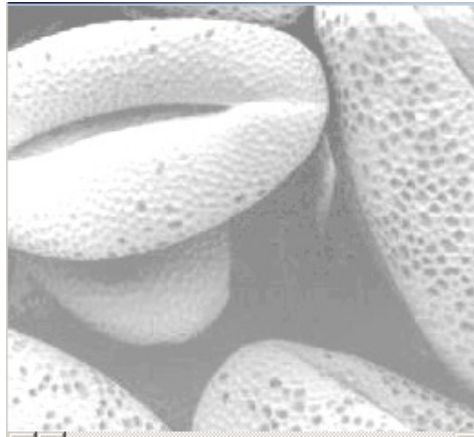
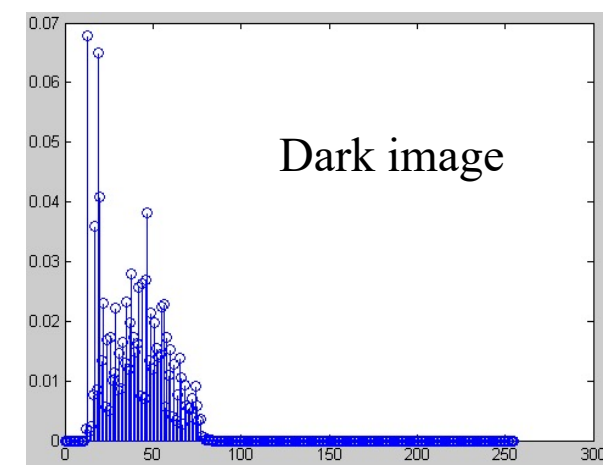
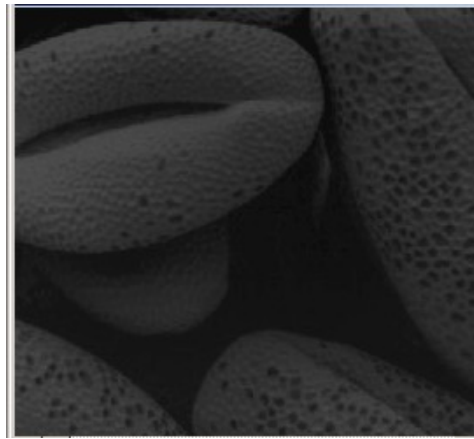
Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Adaptive threshold	.5
Image Histogram	.6
Otus method	.7
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

Histogram Equalization

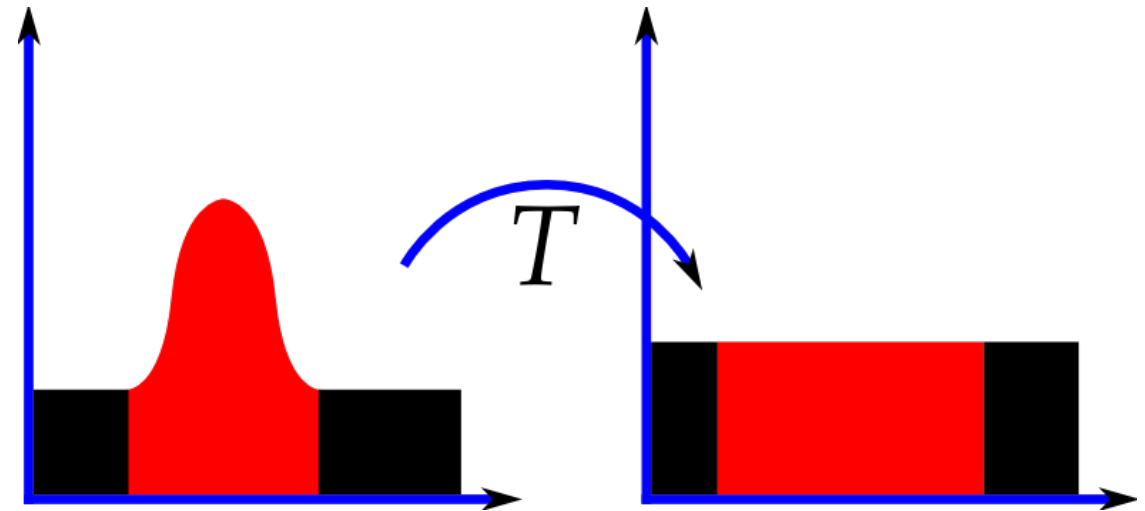
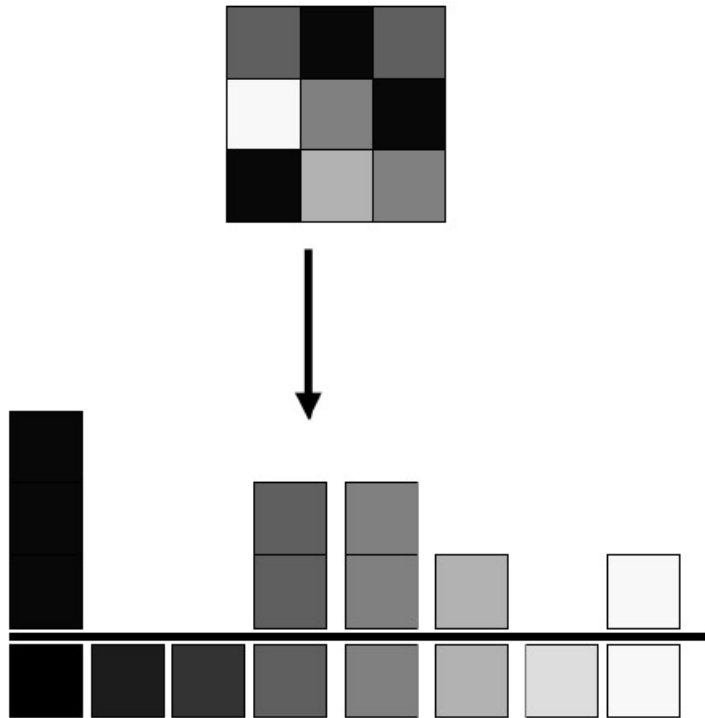
ההשוואה היסטוגרמה משמשת כדי
לשפר את הניגודיות (קונטרסט).

היסטוגרמה – דוגמא נוספת



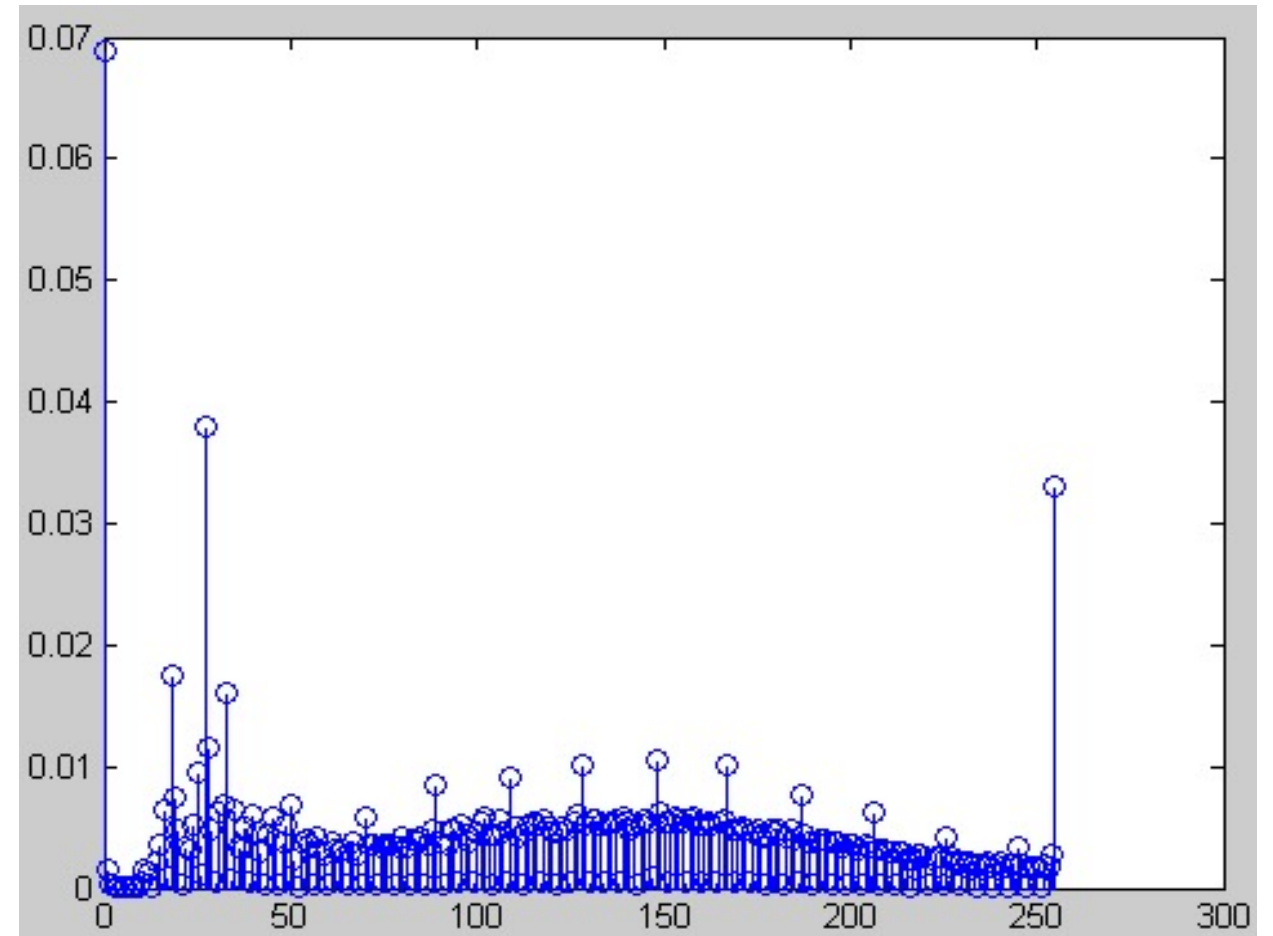
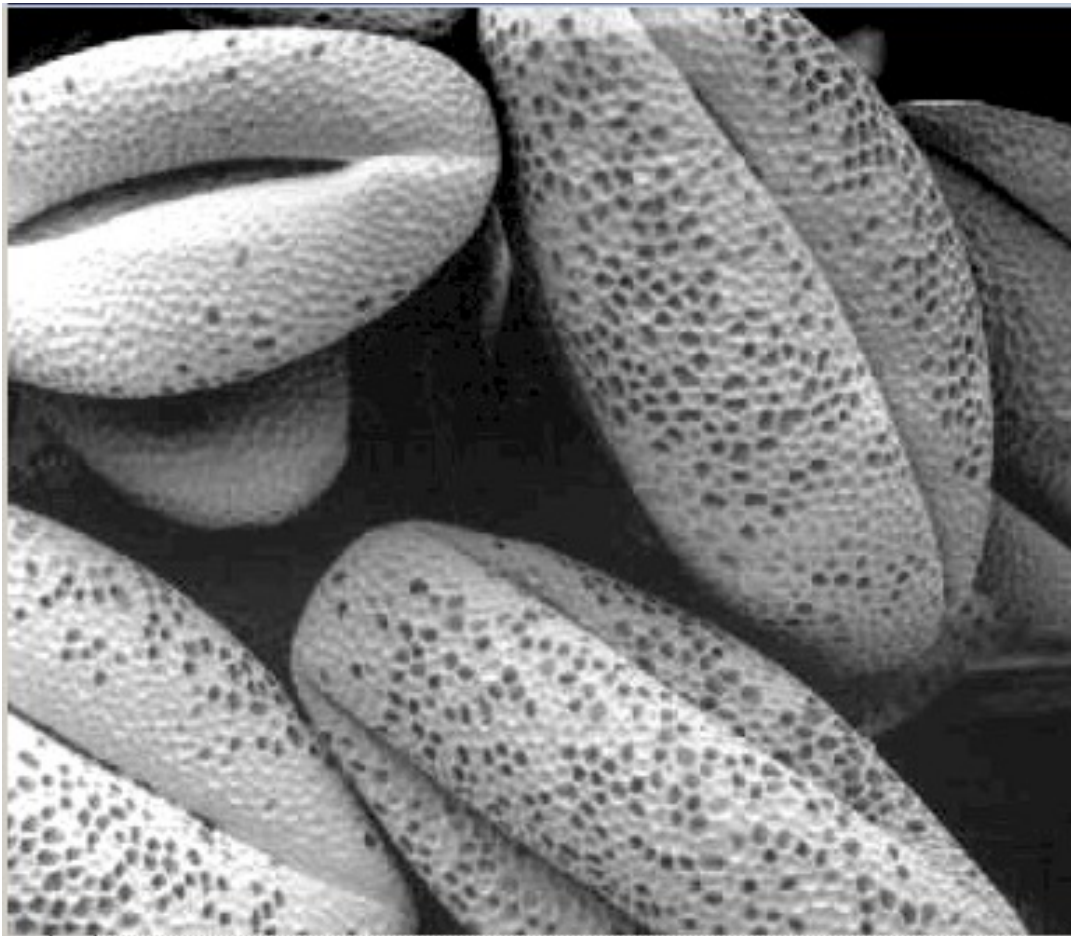
Histogram Equalization

ההשוואה היסטוגרמה משמשת כדי
לשפר את הניגודיות (קונטרסט).



זו המטרה:

תמונה עם קונטרסט טוב \ קונטרסט גבוהה



ההסטוגרמה - בצורה פורמלית:

- אנו עוסקים בתמונות אפור
- כל פיקסל הוא מספר בתחום 0 עד 1
- נחלק את הקטע $[0,1]$ ל- L חתיכות קטנות ($L=256$)
- נסמן את מרכזי הקטעים (שהם רמות אפור) כ- $r_0, r_1, \dots, r_{L-1}$
- נסמן את מספר הפיקסלים שרמת האפור שלהם bin ה- r_k כ- n_k .
- ההיסטוגרמה של התמונה היא פונקציה המוגדרת כך: $p(r_k) = \frac{n_k}{n}$

זה למעשה
ההסתברות
לפיקסל שרמת
האפור r_k

- כאשר n הוא מספר הפיקסלים בתמונה.

Histogram equalization - code

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

img = cv2.imread('origin.jpg',0)
if (img is None):
    print("failed to read the image")
    exit

plt.imshow(img,cmap='gray')
plt.title(" Orig Image ")
plt.show()
```

Histogram equalization

`numpy.histogram(a, bins=10, range=None, ...)`

```
hist, bins = np.histogram(img.flatten(), 256, [0, 256])
```

```
cdf =
```

```
cdf_n
```

```
plt.pl
```

```
plt.his
```

```
plt.xli
```

```
plt.legend(('cdf', 'histogram'), loc = 'upper left')
```

```
plt.show()
```

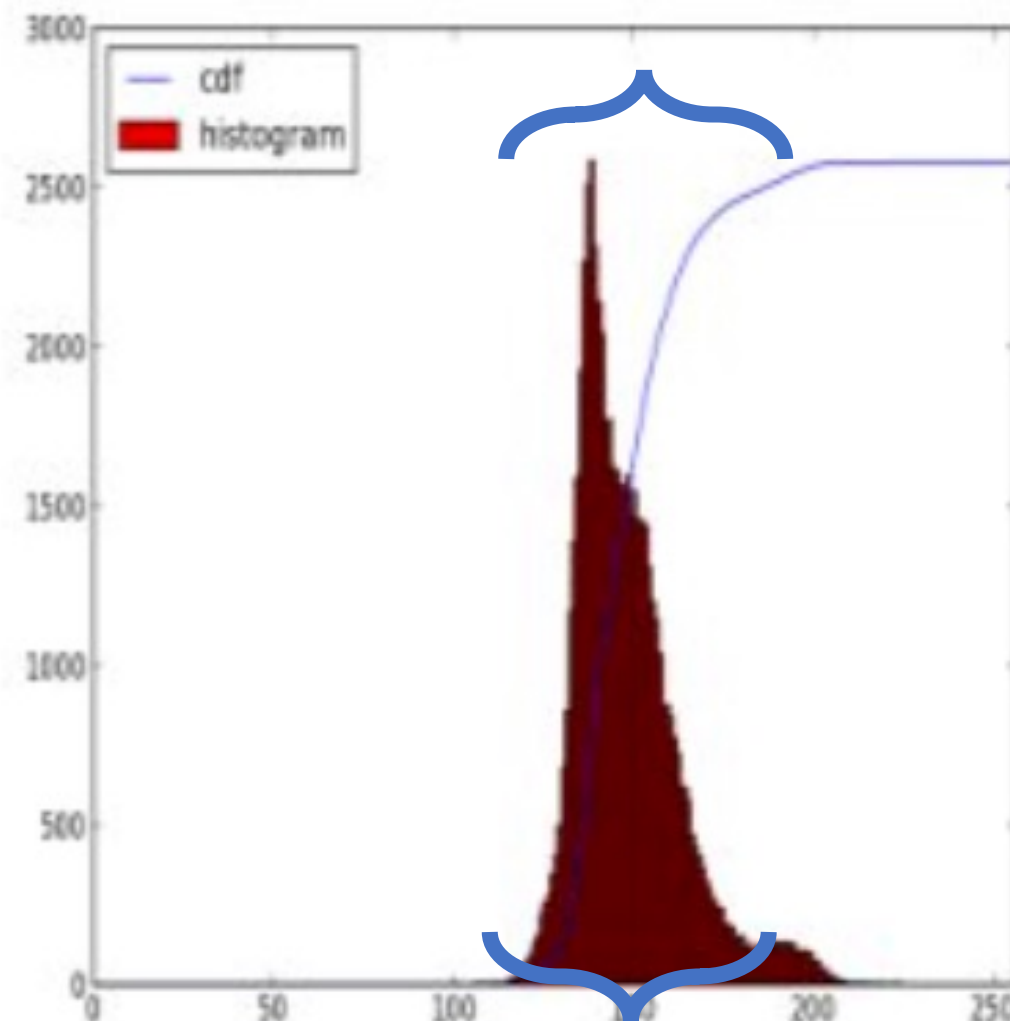
```
# create an array
one_D_arr = np.array(np.arange(10))

print(one_D_arr)
# [0 1 2 3 4 5 6 7 8 9]

# cumulative sum
cumsum = np.cumsum(one_D_arr)

print(cumsum)
# array([ 0, 1, 3, 6, 10, 15, 21, 28, 36, 45])
```

נרצה לנרמל – להשתמש בקטע הזה,
אבל בכל טווח גווני האופור

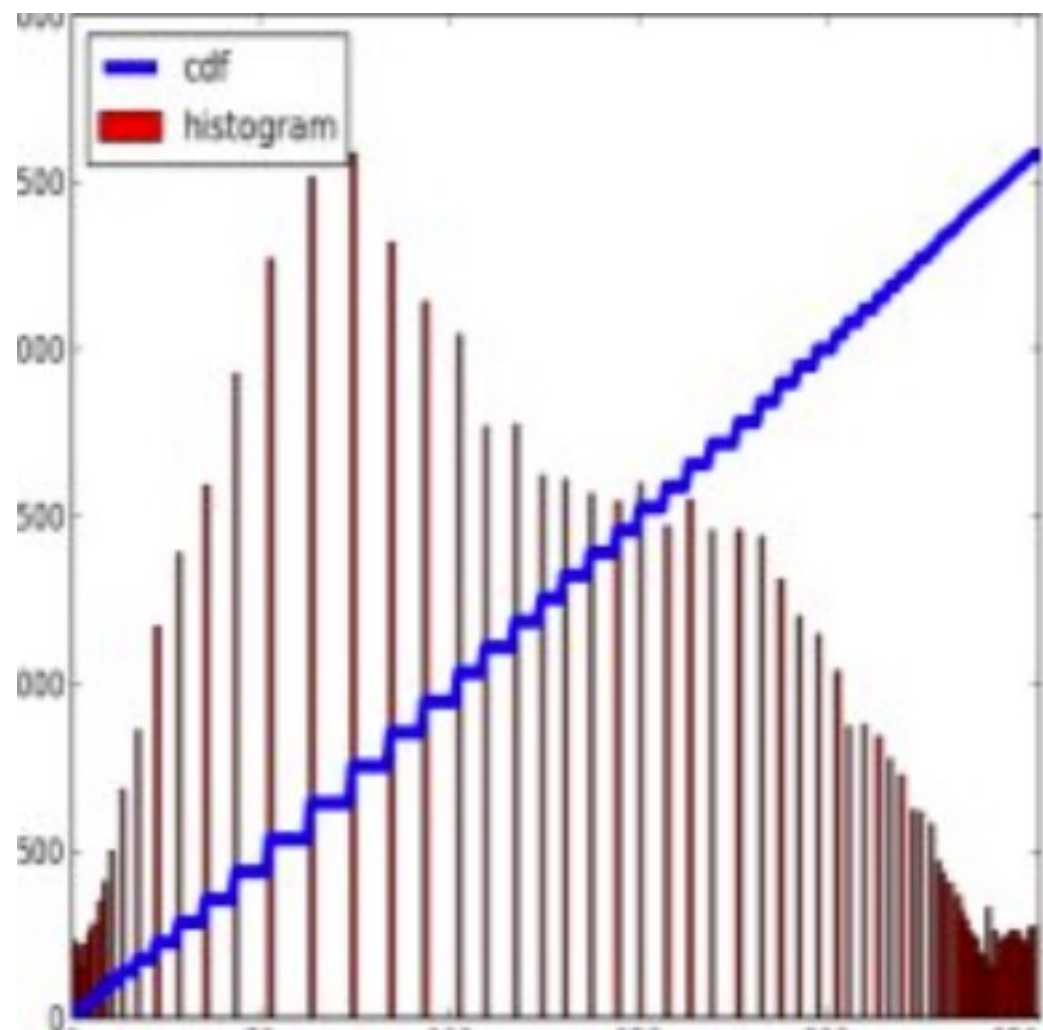


https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_histograms/py_histogram_equalization/py_histogram_equalization.html

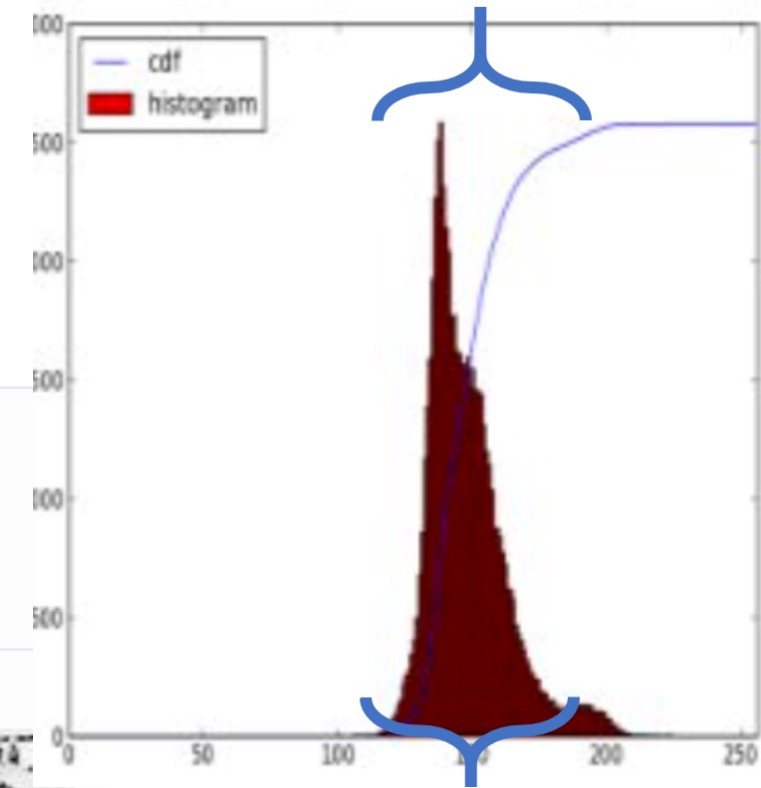
Histogram equalization

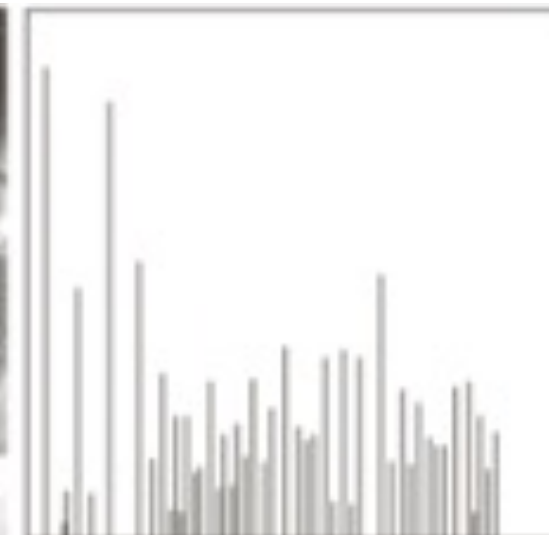
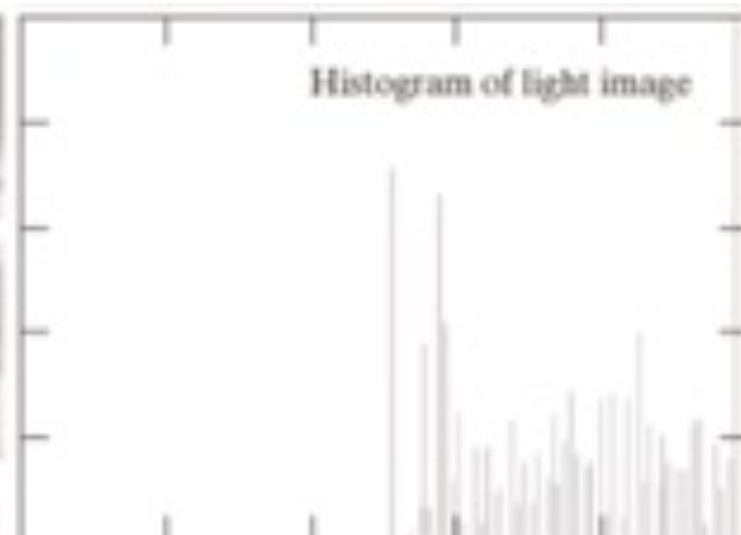
```
cdf_m = np.ma.masked_equal(cdf,0)
cdf_m = (cdf_m - cdf_m.min())*255/(cdf_m.max()-cdf_m.min())
cdf = np.ma.filled(cdf_m,0).astype('uint8')
```

```
img2 = cdf[img]
plt.imshow(img2,cmap='gray')
plt.show()
```

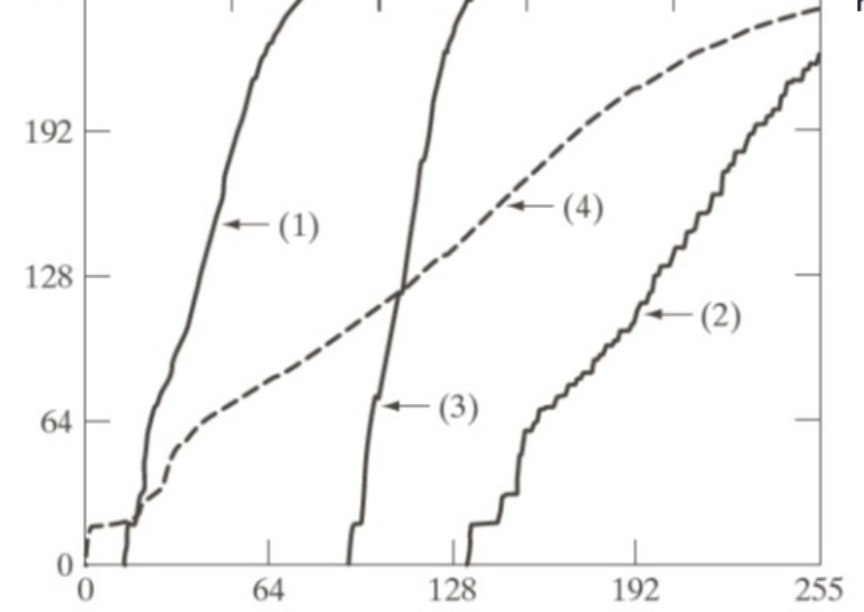
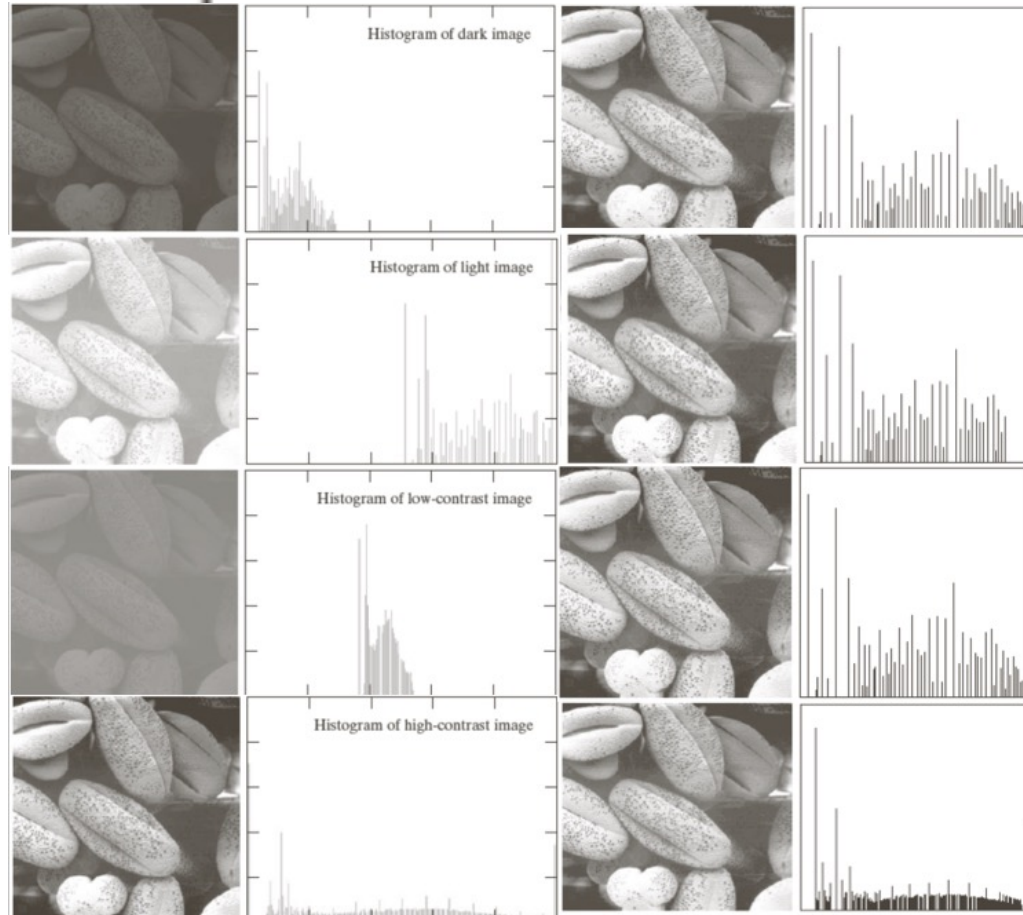



```
img = cv2.imread('wiki.jpg',0)
equ = cv2.equalizeHist(img)
res = np.hstack((img,equ)) #stacking images side-by-side
cv2.imwrite('res.png',res)
```





Transformation functions for histogram equalization.



example

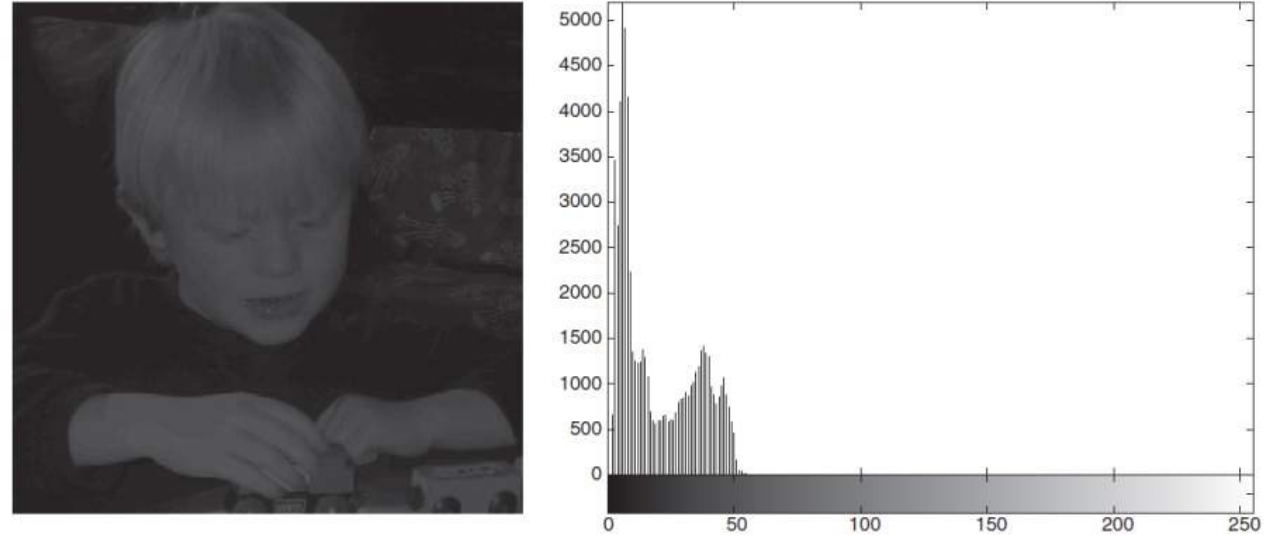


FIGURE 4.20 *The darkened version of `engineer.tif` and its histogram.*

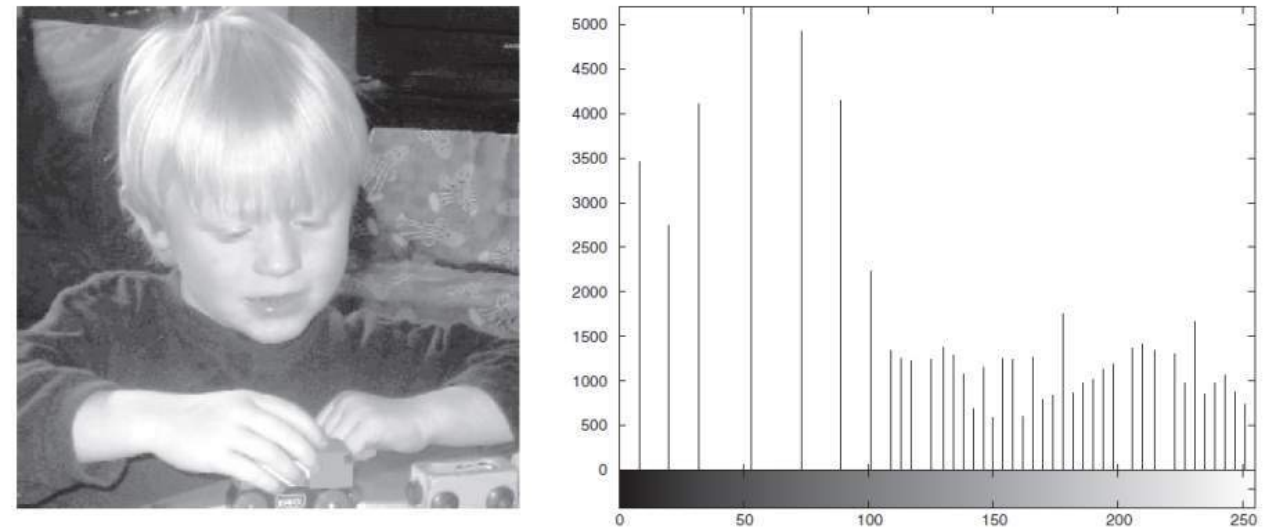
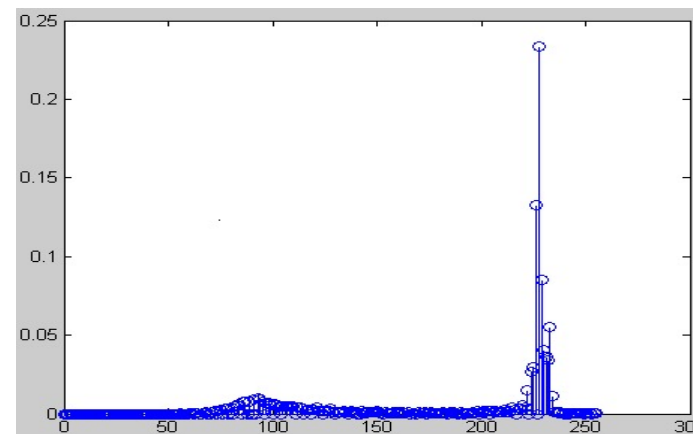
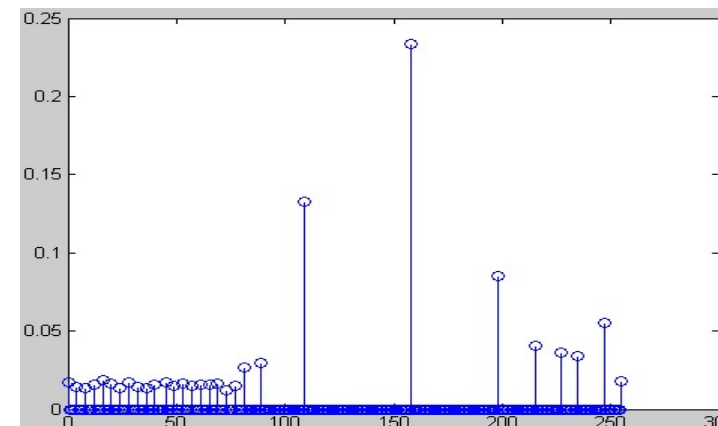
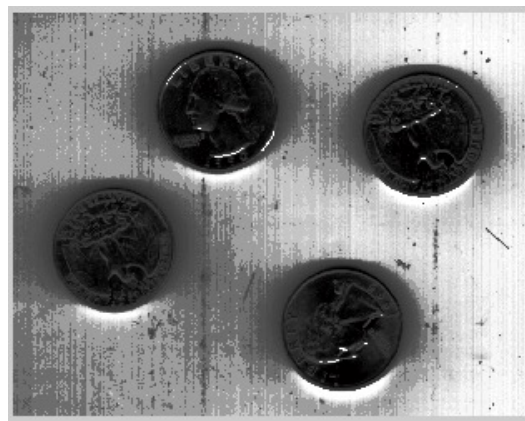


FIGURE 4.21 *The image from Figure 4.20 equalized and its histogram.*

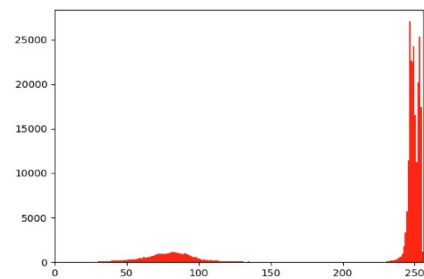
מה יקרה לתמונה הזאת?



ההשוואה היסטוגרמה זה לא תהליך מובטח. וייתכנו מקרים מסוימים שדווקא השוואת היסטוגרמה תהרוס את הקונטרסט.



מה יקרה לתמונה הזאת?



ההשוואה היסטוגרמה זה לא תהליך מובטח. וייתכנו מקרים מסוימים שדווקא השוואת היסטוגרמה תהרוס את הקונטרסט.

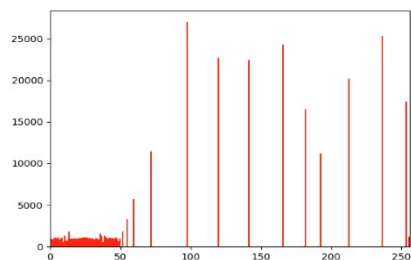


Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Adaptive threshold	.5
Image Histogram	.6
Otus method	.7
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

Local histogram equalization

**Adaptive histogram equalization
(CLAHE)**

Contrast-limited adaptive histogram equalization (CLAHE)

שיפור לוקאלי

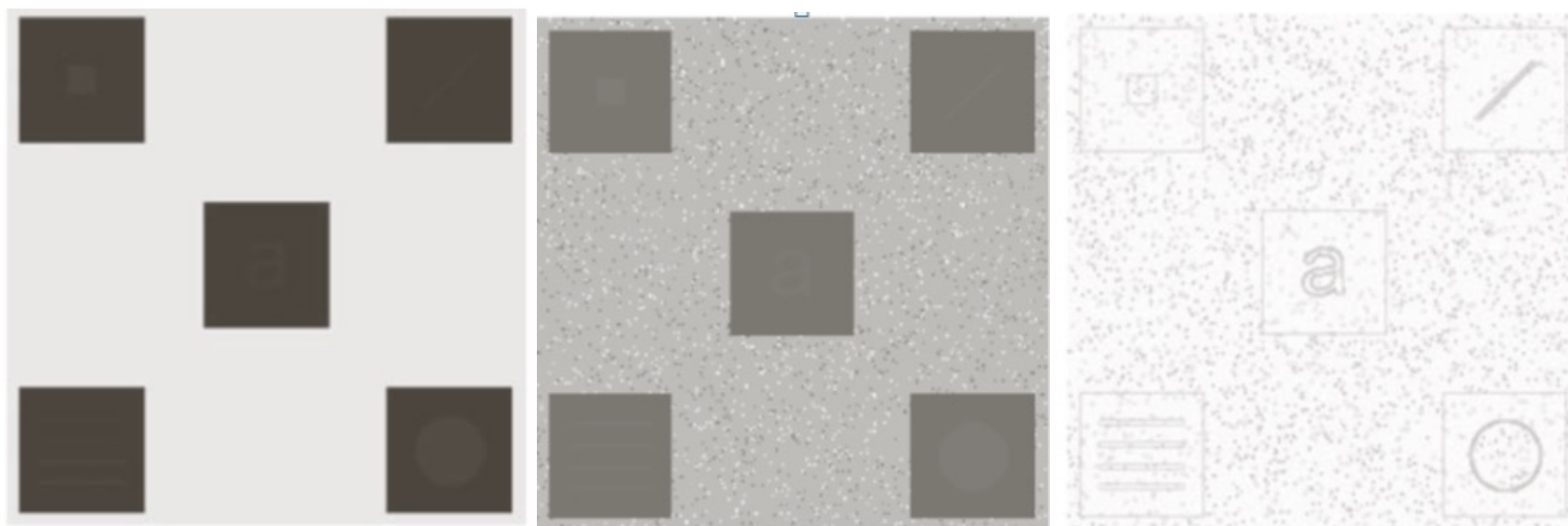
הפעולות שהגדרנו עד כה הן גלובליות. לפעמים השינויים הנדרשים הם מקומיים - מספר הפיקסלים בכל אזור לא ישפיע על ההסטוגרמה הכללית.

משמעותי במיוחד כאשר יש אזור בהיר ואזור כהה ובשניהם נדרש עיבוד.

אלגוריתם: מחליפים כל פיקסל בערך המתאים משיווי ההסטוגרמה בחלון סביבו.

שימו לב שלא צריך לחשב את כל ההסטוגרמה. החלונות של פיקסלים שכנים חופפים וניתן לנצל זאת.

Local histogram equalization



Local histogram equalization

```
import numpy as np
import cv2

img = cv2.imread('tsukuba_1.png',0)

# create a CLAHE object (Arguments are optional).
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
c11 = clahe.apply(img)

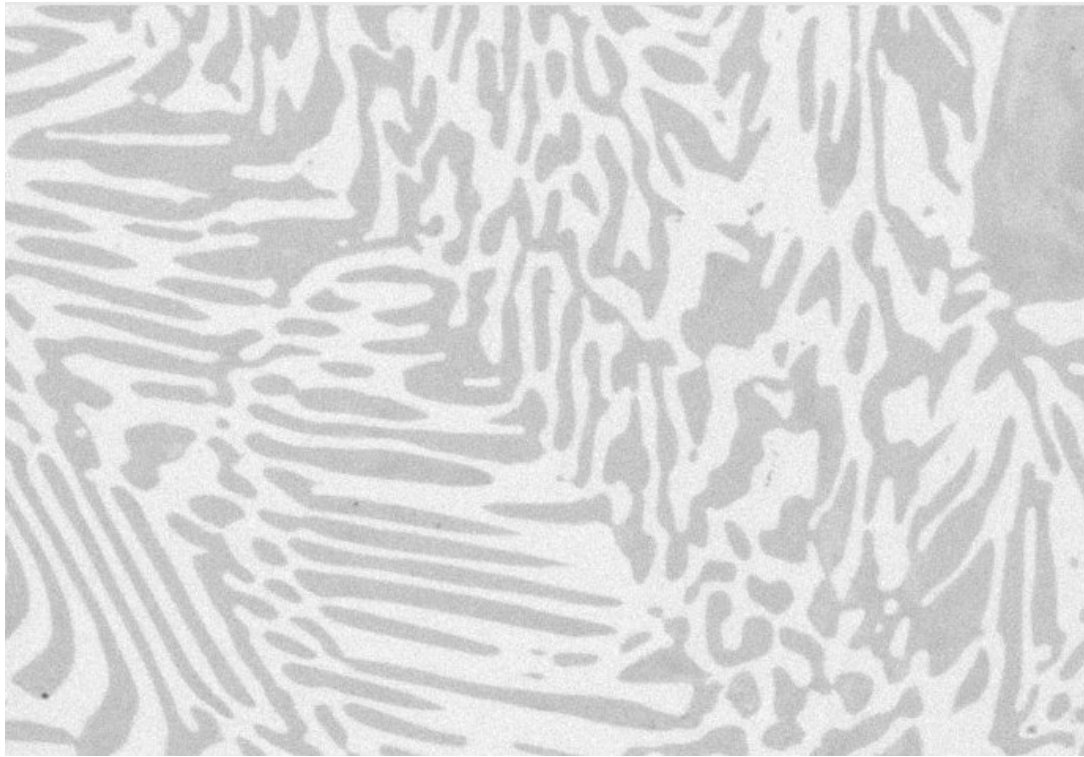
cv2.imwrite('clahe_2.jpg',c11)
```

Class exercise

בעזרת השיטות שלמדנו השיעור – נקו כמיטב יכולתכם את התמונה (שעלתה לתרגיל 2) לקבלת תמונה כמה שיותר שלמה ונקיה של הרקמות

Class exercise

Input



The desired output
should look like this...



Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Adaptive threshold	.5
Image Histogram	.6
Otus method	.7
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4

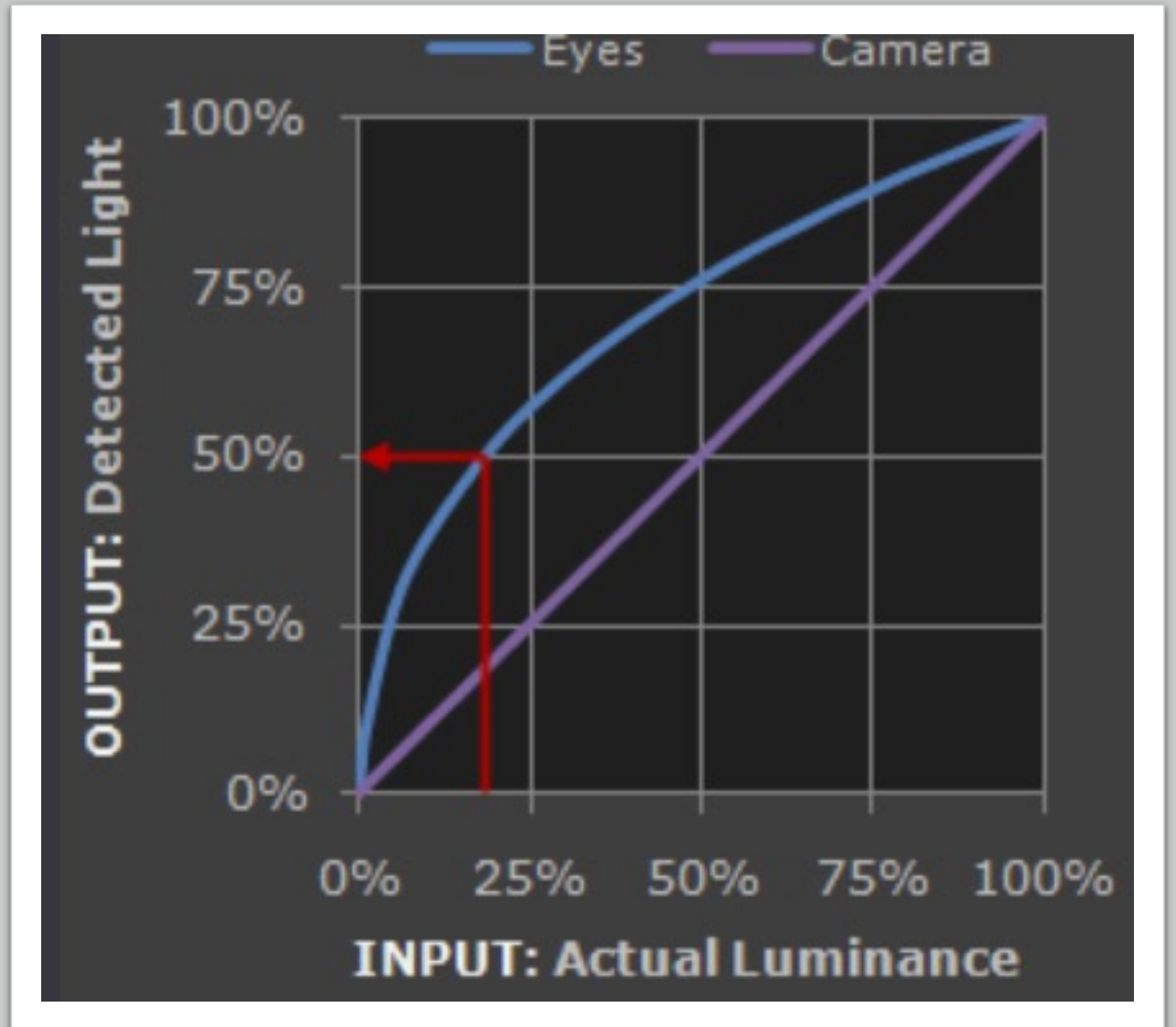
Gamma correction

הראינו כיצד אנו משנים קונטרסט – כעת נעבור להראות שינוי בהירות. בהירות היא בעצם רמת האפור הממוצעת בתמונה. לכן, בחירה אינטואיטיבית לפעולת שינוי בהירות תהיה הוספת ערך קבוע לרמת האפור. בחירה כזו אינה מוצלחת בדרך כלל כיוון שהיא גורמת להוצאת הערכים מהתחום המותר. כתחליף, מקובל להשתמש בתיקון גאמה (Gamma Correction), אשר נראית כך:

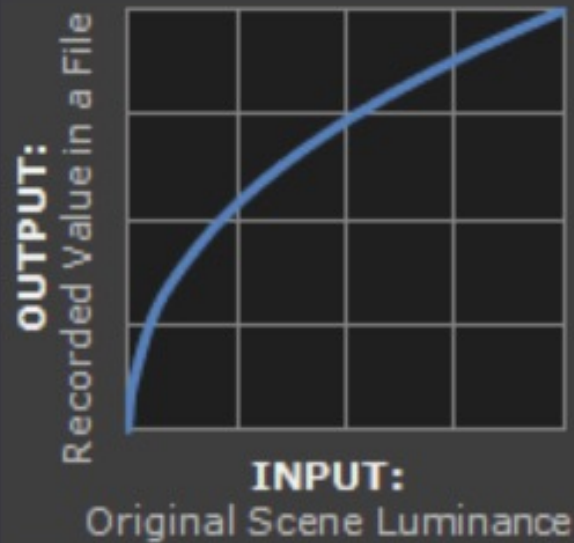
https://en.wikipedia.org/wiki/Gamma_correction

<https://www.cambridgeincolour.com/tutorials/gamma-correction.htm>

Gamma correction



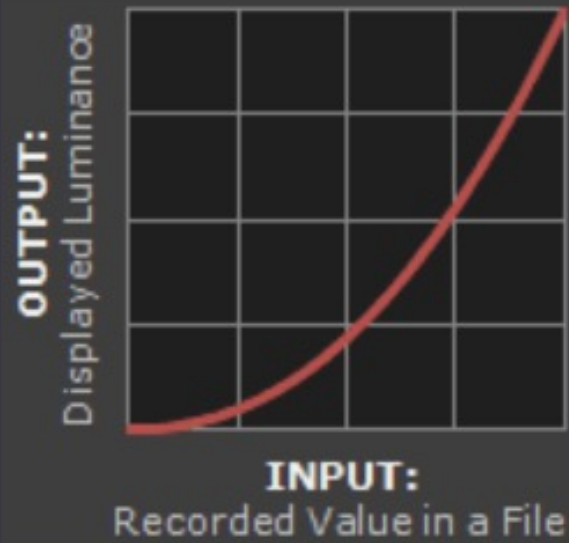
RAW Camera Image
is Saved as a JPEG File



1. Image File Gamma

+

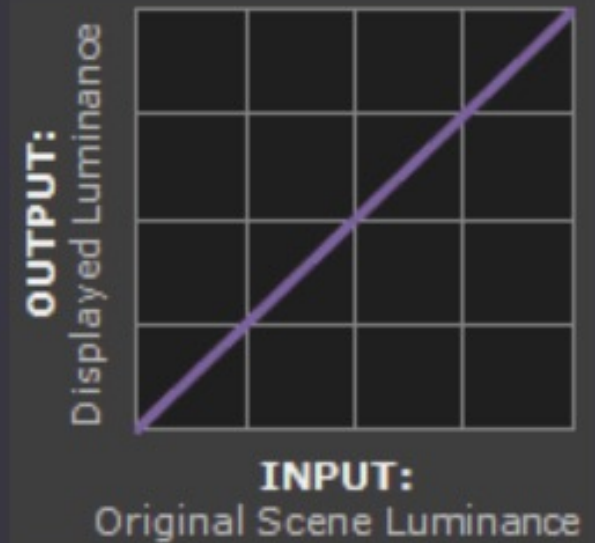
JPEG is Viewed
on a Computer Monitor



2. Display Gamma

=

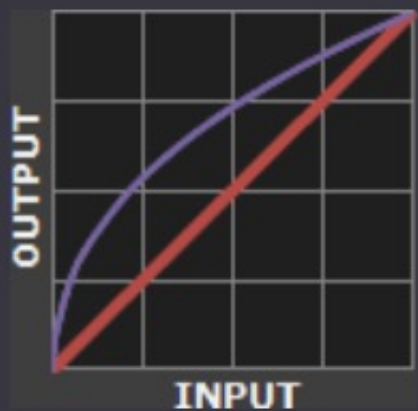
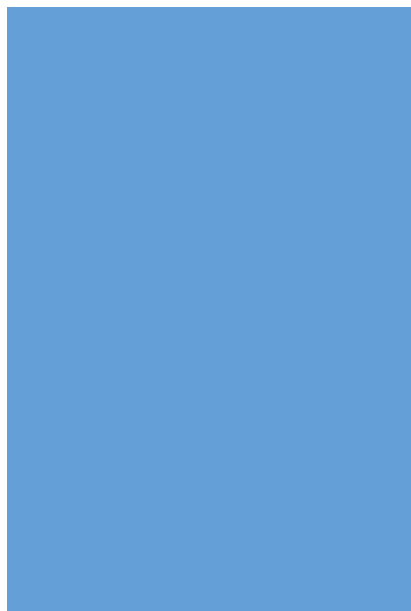
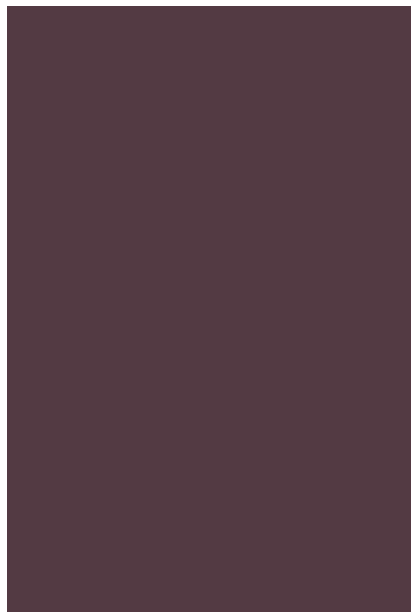
Net Effect



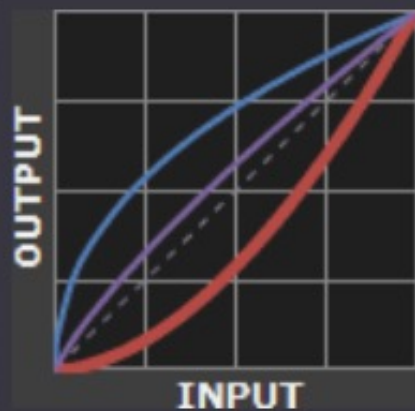
3. System Gamma

1. Depicts an image in the sRGB color space (which encodes using a gamma of approx. 1/2.2).

2. Depicts a display gamma equal to the standard of 2.2



Display Gamma 1.0



Display Gamma 1.8



Display Gamma 2.2



Display Gamma 4.0



Diagrams assume that your display has been calibrated to a standard gamma of 2.2. Recall from before that the image file gamma (—) plus the display gamma (—) equals the overall system gamma (—). Also note how higher gamma values cause the red curve to bend downward.

Gamma correction

$$y = \left(\frac{x - a}{b - a} \right)^\gamma (d - c) + c$$

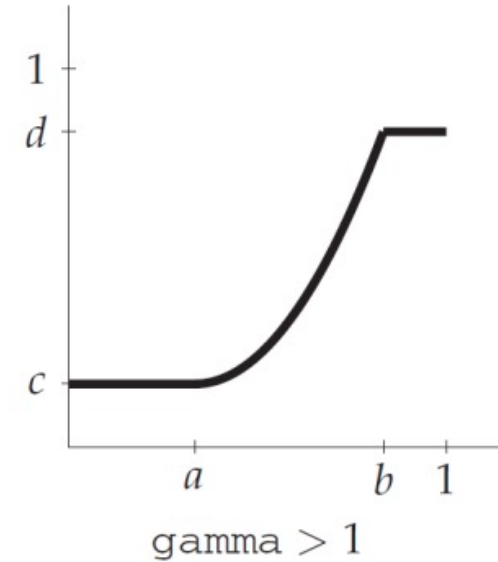
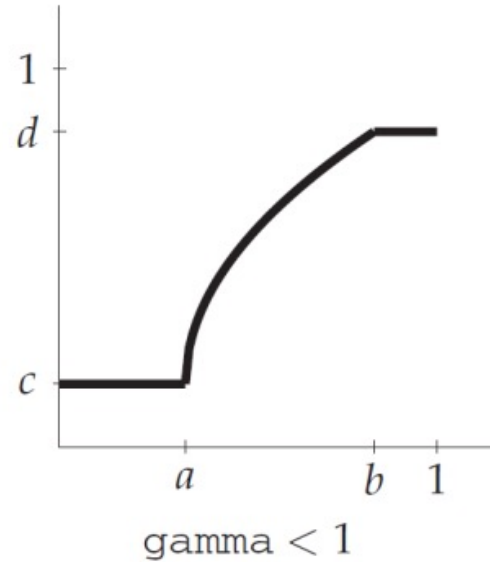


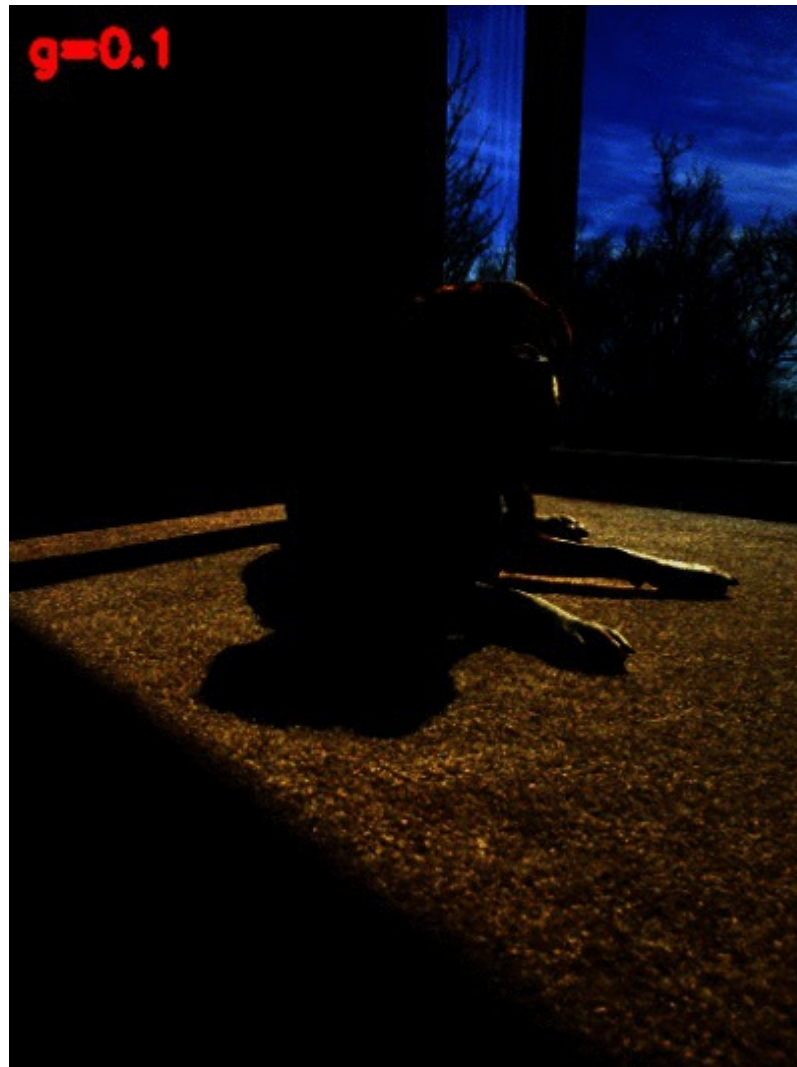
FIGURE 4.11 The imadjust function with gamma not equal to 1.

$$y = \left(\frac{x}{255} \right)^\gamma \cdot 255$$

מקרה פרטי בו:

$a=c=0$ •

$b=d=255$ •



עיבוד סיפרתי של תמונות: מבוא

```
# import the necessary packages
from __future__ import print_function
import numpy as np
import argparse
import cv2

def adjust_gamma(image, gamma=1.0):
    # build a lookup table mapping the pixel values [0, 255] to
    # their adjusted gamma values
    invGamma = 1.0 / gamma
    table = np.array([((i / 255.0) ** invGamma) * 255
        for i in np.arange(0, 256)]).astype("uint8")

    # apply gamma correction using the lookup table
    return cv2.LUT(image, table)
```

```
# loop over various values of gamma
for gamma in np.arange(0.0, 3.5, 0.5):
    # ignore when gamma is 1 (there will be no change to the image)
    if gamma == 1:
        continue

    # apply gamma correction and show the images
    gamma = gamma if gamma > 0 else 0.1
    adjusted = adjust_gamma(original, gamma=gamma)
    cv2.putText(adjusted, "g={}".format(gamma), (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 3)
    cv2.imshow("Images", np.hstack([original, adjusted]))
    cv2.waitKey(0)
```

Image normalization

image normalization	.0
פעולות על רמות אפור (דרגות אפור)	.1
מושגי בסיס:	.2
Brightness	.1
contrast	.2
Threshold operation	.3
Bi linear interpolation	.4
Adaptive threshold	.5
Image Histogram	.6
Otus method	.7
Type of image normalization	.3
Simple normalization	.1
Histogram equalization	.2
Contrast Limited Adaptive Histogram Equalization – CLAHE	.3
Gamma Correction	.4
Color image Calibration	.4